

NightOut

Authors: Riccardo Masarin, Manuel Mannucci, Alessandro Forlani

Project: Software Engineering for Automation — Project A

Version: 2.0

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Definitions, Acronyms and Abbreviations
 - 1.3 Revision history
 - 1.4 Document Structure
2. Architectural Design
 - 2.1 Overview
 - 2.2 Component View
 - 2.3 Class View
 - 2.4 Runtime View
 - 2.5 Selected Architectural Styles and Patterns
3. User Interface Design
 - 3.1 UI Overview
 - 3.2 Main User Interfaces
 - 3.3 Venue Manager Interface
4. Requirements Traceability
 - 4.1 Traceability Matrix
 - 4.2 Traceability Discussion
5. Implementation, Integration and Test Plan
 - 5.1 Implementation Order
 - 5.2 Integration Plan
 - 5.3 Testing Strategy
 - 5.4 Test Data
6. References

1. Introduction

1.1 Purpose

This document refines the requirements defined in the Requirements Analysis and Specification Document and describes how the system is currently structured. In particular, it presents the software architecture, the main components, the relevant class diagrams, the runtime interactions among components, the user interface design, the mapping between requirements and design elements, and the implementation, integration, and testing plan.

In particular, the document presents:

- the client-server and layered architecture of the system;
- the main frontend and backend components;
- the implemented domain model and persistence structure;
- the design patterns effectively used at runtime;
- the main interactions between the React frontend and the Spring Boot backend;
- the implemented user, venue manager, and PR manager interfaces;
- the traceability between functional requirements and implementation elements;
- the current implementation, integration, testing, and dataset status;
- the main prototype limitations and known implementation differences.

NightOUT is implemented as a React and TypeScript web client communicating through REST APIs with a Java Spring Boot backend. The backend uses Spring Data JPA and an in-memory H2 database for persistence during the execution of the prototype.

1.2 Definitions, Acronyms and Abbreviations

User

A registered person using the system to discover events, reserve spots, join pregame rooms, and interact with social features.

Venue

A physical place, such as a club, bar, lounge, or live music venue, where nightlife events take place.

Venue Manager

A user whose role is VENUE_MANAGER. Venue managers can manage their assigned venues, create and update events, manage promotions, inspect tickets, and access dashboard information.

PR Manager

A user whose role is PR_MANAGER. PR managers can access assigned event information, attributed ticket data, commissions, and PR-specific dashboard information.

Event

A nightlife activity associated with a venue.

Ticket

A persistent record representing a user's request to attend an event. A ticket progresses through a lifecycle that may include pending, confirmed, waiting-list, cancelled, and expired states. Confirmed tickets contain a digital QR payload used by the prototype as an admission confirmation.

Waiting List

A list of users waiting for a spot in an event that reached its full capacity.

Pregame Room

A temporary group linked to an event, used by users to coordinate before attending the event.

Friendship

A social relationship between two users created through a request workflow. A friendship request may be pending, accepted, or rejected.

Notification

A message generated by the system to inform a user about relevant updates.

REST API

Representational State Transfer Application Programming Interface.

DTO

Data Transfer Object.

UI

User Interface.

1.3 Revision History

Version	Date	Description
1.0	Initial design date	Initial planned software architecture and implementation proposal
2.0	21 July 2026	Updated as-built Design Document aligned with the current React, Spring Boot, JPA and H2 implementation

1.4 Document Structure

Section 1 introduces the purpose, terminology, revision history, and structure of the document.

Section 2 describes the as-built architecture of NightOUT, including the client-server structure, backend layers, implemented components, domain classes, runtime interactions, and design patterns.

Section 3 presents the current user interfaces and route organization for normal users, venue managers, and PR managers.

Section 4 provides the requirements traceability matrix, that maps the functional requirements defined in the Requirements Analysis and Specification Document to the corresponding implementation elements and reports their implementation status.

Section 5 describes the current implementation and integration status, the automated tests actually available, the additional tests still required, the prototype dataset, and the main known limitations.

Section 6 lists the references used in the document.

2. Architectural Design

2.1 Overview

NightOUT follows a **client-server architecture**.

The client side is a mobile-oriented React and TypeScript frontend. It is responsible for presentation, navigation, user interaction, and local interface state. The frontend communicates with the backend through an Axios-based REST API client and exchanges JSON Data Transfer Objects.

Three main roles access the application:

- normal users;
- venue managers;
- PR managers.

The frontend does not contain the authoritative application data. Operational information concerning users, venues, events, tickets, pregame rooms, friendships, promotions, and notifications is retrieved from the backend through REST API calls.

The server side is implemented in Java using Spring Boot. It exposes the REST endpoints, validates incoming requests, executes the application and domain logic, coordinates persistence operations, and returns DTO responses to the frontend.

The database stores users, venue managers, venues, events, reservations, pregame rooms, promotions, notifications, social relations, and event participation information.

The backend follows a layered architecture:

1. **REST Controller Layer**
Receives HTTP requests, validates request DTOs, invokes the appropriate application services, and returns response DTOs.
2. **Application Service Layer**
Implements use-case management, transaction boundaries, business rules, and coordination among domain objects, recommendation strategies, notification mechanisms, and persistence components.
3. **Domain and Business Behavior Layer**
Contains the JPA entities, enums, ticket lifecycle states, recommendation strategies, and domain-related events used by the application.
4. **Data Mediation Layer**
Contains multiple focused data mediators. Each mediator coordinates the

repositories required by a specific application area, such as tickets, events, users, pregames, recommendations, or manager dashboards.

5. **Repository Layer**

Contains Spring Data JPA repositories responsible for persistent entity access, queries, counts, and data updates.

6. **Persistence Layer**

Uses an in-memory H2 relational database. Hibernate is configured with a create-drop policy, meaning that data remains available while the backend process is running but is recreated when the backend restarts.

The normal backend dependency direction is:

REST Controllers → Application Services → Focused Data Mediators → Spring Data JPA Repositories → H2 Database.

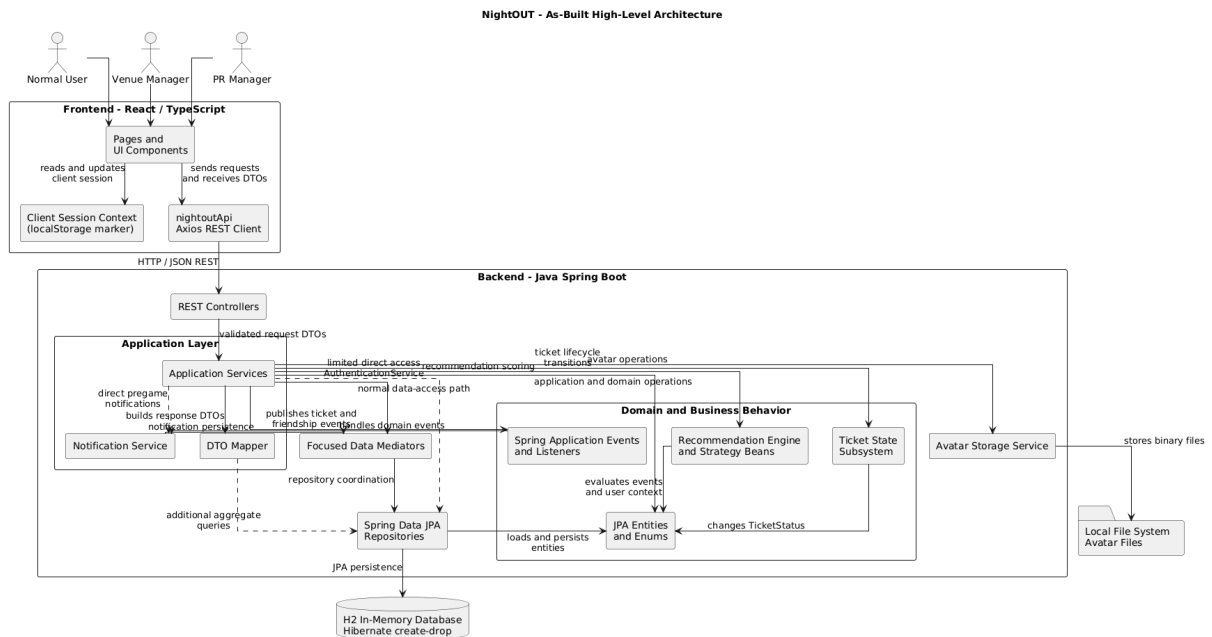
Application services also collaborate with several specialized runtime components. The ticket lifecycle is implemented through concrete State objects selected by a state factory. Personalized recommendations are produced by a recommendation engine that evaluates multiple Spring-managed recommendation strategies. Ticket and friendship notifications use Spring application events and listeners, while some pregame notification operations currently invoke the notification service directly.

The current implementation contains two limited exceptions to the main data-access direction. `AuthenticationService` accesses `AppUserRepository` directly, while `NightOutMapper` performs additional repository queries when constructing aggregate event DTOs.

Most persistent application data is stored through JPA in H2. Avatar binary files are stored in the local file system, while their corresponding resource URLs are stored in the user entity.

Authentication in the current prototype is intentionally lightweight. The backend verifies the submitted credentials, while the frontend stores a session marker in browser `localStorage`. The prototype does not currently use Spring Security, server-side sessions, or token-based authentication. Consequently, backend operations still rely on user and manager identifiers supplied by the client.

The general architecture is shown below.



@startuml

title NightOUT - As-Built High-Level Architecture

top to bottom direction

skinparam componentStyle rectangle

skinparam packageStyle rectangle

skinparam shadowing false

skinparam linetype ortho

actor "Normal User" as User

actor "Venue Manager" as VenueManager

actor "PR Manager" as PRManager

package "Frontend - React / TypeScript" {

component "Pages and\nUI Components" as UI

component "Client Session Context\n(localStorage marker)" as Session

component "nightoutApi\nAxios REST Client" as ApiClient

UI --> Session : reads and updates\nclient session

UI --> ApiClient : sends requests\nand receives DTOs

}

package "Backend - Java Spring Boot" {

component "REST Controllers" as Controllers

package "Application Layer" {

```

component "Application Services" as Services
component "Notification Service" as NotificationService
component "DTO Mapper" as Mapper
}

package "Domain and Business Behavior" {
    component "JPA Entities\nand Enums" as Domain
    component "Ticket State\nSubsystem" as TicketState
    component "Recommendation Engine\nand Strategy Beans" as
RecommendationEngine
    component "Spring Application Events\nand Listeners" as DomainEvents
}

component "Focused Data Mediators" as Mediators

component "Spring Data JPA\nRepositories" as Repositories

component "Avatar Storage Service" as AvatarStorage
}

database "H2 In-Memory Database\nHibernate create-drop" as H2

folder "Local File System\nAvatar Files" as FileSystem

User --> UI
VenueManager --> UI
PRManager --> UI

ApiClient --> Controllers : HTTP / JSON REST

Controllers --> Services : validated request DTOs
Services --> Mapper : builds response DTOs
Services --> Domain : application and domain operations

Services --> Mediators : normal data-access path
NotificationService --> Mediators : notification persistence
Mediators --> Repositories : repository coordination
Repositories --> H2 : JPA persistence

Repositories --> Domain : loads and persists\nentities

Services --> TicketState : ticket lifecycle\ntransitions
TicketState --> Domain : changes TicketStatus

Services --> RecommendationEngine : recommendation scoring
RecommendationEngine --> Domain : evaluates events\nand user context

Services --> DomainEvents : publishes ticket and\nfriendship events
DomainEvents --> NotificationService : handles domain events
Services ..> NotificationService : direct pregame\nnotifications

Services --> AvatarStorage : avatar operations
AvatarStorage --> FileSystem : stores binary files

```

```
Services ..> Repositories : limited direct access\nAuthenticationService  
Mapper ..> Repositories : additional aggregate\nqueries
```

@enduml

2.2 Component View

The component view describes the main functional areas of the NightOUT implementation and the dependencies among the frontend, backend application components, data-access infrastructure, and persistence layer.

A component in this view represents a cohesive group of controllers, services, domain objects, mediators, repositories, and frontend pages that collaborate to provide a specific group of functionalities. Therefore, each component does not necessarily correspond to a single Java class or React component.

The React frontend is organized into role-specific pages for normal users, venue managers, and PR managers. All frontend areas communicate with the backend through the shared `nightoutApi` Axios client.

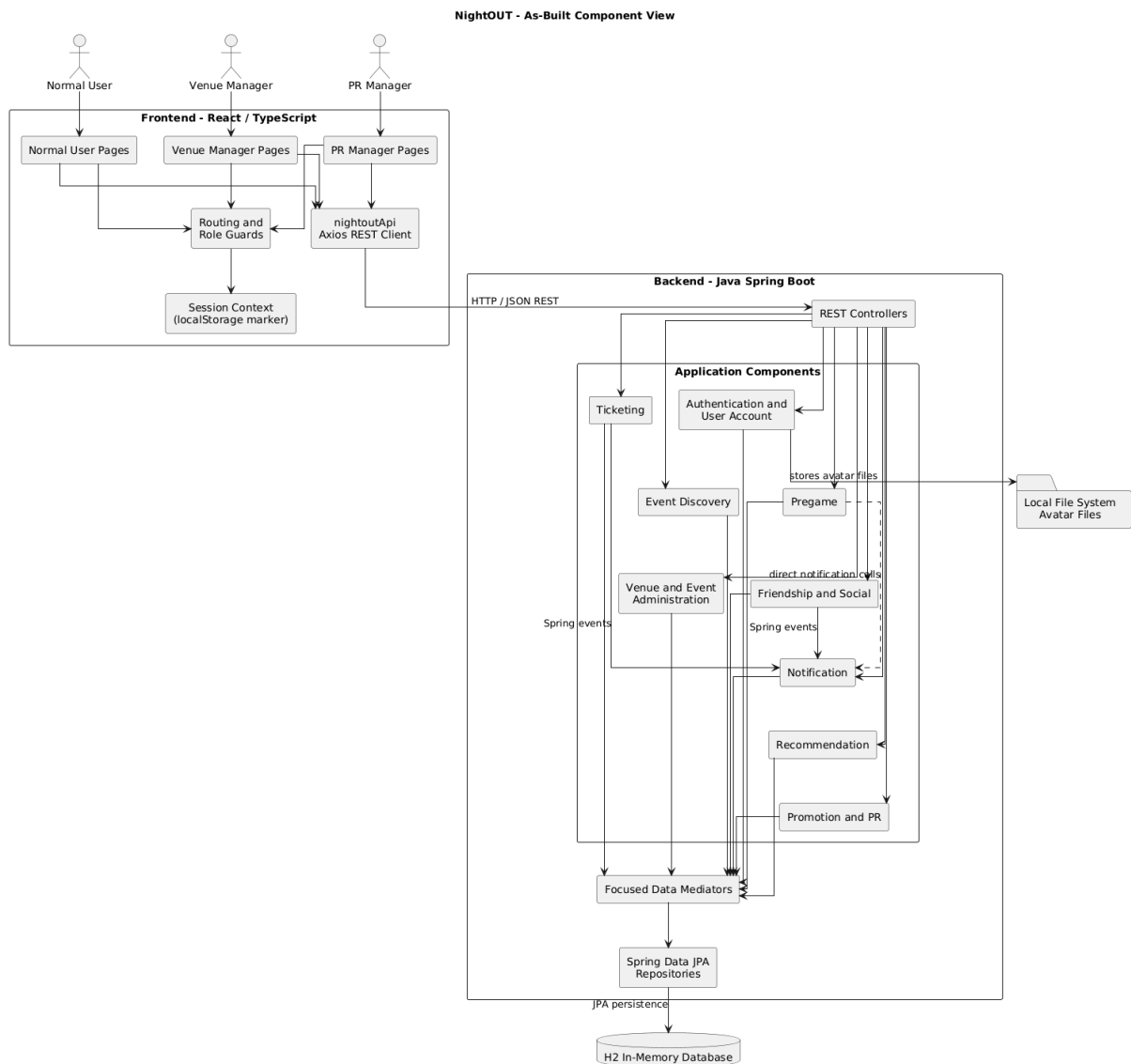
The Spring Boot backend is divided into the following main application components:

1. Authentication and User Account Component;
2. Event Discovery Component;
3. Venue and Event Administration Component;
4. Ticketing Component;
5. Pregame Component;
6. Friendship and Social Component;
7. Notification Component;
8. Recommendation Component;
9. Promotion and PR Component;
10. Data Access and Persistence Component.

The backend application components primarily access persistent data through focused data mediators. The mediators coordinate Spring Data JPA repositories, which store the application entities in the H2 database.

Some components also collaborate through specialized runtime mechanisms. Ticket lifecycle operations use the State pattern, recommendation scoring uses multiple Strategy implementations, and ticket and friendship notifications use Spring application events and listeners.

The component diagram is shown below.



@startuml
title NightOUT - As-Built Component View

top to bottom direction

skinparam componentStyle rectangle
skinparam packageStyle rectangle
skinparam shadowing false
skinparam linetype ortho
skinparam defaultTextAlignment center

actor "Normal User" as User
actor "Venue Manager" as VenueManager
actor "PR Manager" as PRManager

package "Frontend - React / TypeScript" {

```
component "Normal User Pages" as UserUI
component "Venue Manager Pages" as ManagerUI
component "PR Manager Pages" as PRUI
```

```
component "Routing and\nRole Guards" as Router
component "Session Context\n(localStorage marker)" as Session
component "nightoutApi\nAxios REST Client" as ApiClient
```

```
User --> UserUI
VenueManager --> ManagerUI
PRManager --> PRUI
```

```
UserUI --> Router
ManagerUI --> Router
PRUI --> Router
```

```
Router --> Session
```

```
UserUI --> ApiClient
ManagerUI --> ApiClient
PRUI --> ApiClient
```

```
}
```

```
package "Backend - Java Spring Boot" {
```

```
    component "REST Controllers" as Controllers
```

```
    package "Application Components" {
```

```
        component "Authentication and\nUser Account" as Auth
        component "Event Discovery" as Discovery
        component "Venue and Event\nAdministration" as VenueAdmin
```

```
        component "Ticketing" as Ticketing
        component "Pregame" as Pregame
        component "Friendship and Social" as Social
```

```
        component "Notification" as Notification
        component "Recommendation" as Recommendation
        component "Promotion and PR" as PromotionPR
```

```
        Auth -[hidden]-> Discovery
        Discovery -[hidden]-> VenueAdmin
```

```
        Ticketing -[hidden]-> Pregame
        Pregame -[hidden]-> Social
```

```
        Notification -[hidden]-> Recommendation
        Recommendation -[hidden]-> PromotionPR
```

```
    }
```

```
    component "Focused Data Mediators" as Mediators
    component "Spring Data JPA\nRepositories" as Repositories
```

```

}

database "H2 In-Memory Database" as H2
folder "Local File System\nAvatar Files" as AvatarFiles

ApiClient --> Controllers : HTTP / JSON REST

Controllers --> Auth
Controllers --> Discovery
Controllers --> VenueAdmin
Controllers --> Ticketing
Controllers --> Pregame
Controllers --> Social
Controllers --> Notification
Controllers --> Recommendation
Controllers --> PromotionPR

Auth --> Mediators
Discovery --> Mediators
VenueAdmin --> Mediators
Ticketing --> Mediators
Pregame --> Mediators
Social --> Mediators
Notification --> Mediators
Recommendation --> Mediators
PromotionPR --> Mediators

Mediators --> Repositories
Repositories --> H2 : JPA persistence

Auth --> AvatarFiles : stores avatar files

Ticketing --> Notification : Spring events
Social --> Notification : Spring events
Pregame --> Notification : direct notification calls

@enduml

```

2.2.1 Authentication and User Account Component

This component manages credential-based login and the information associated with application accounts.

Normal users, venue managers, and PR managers are represented by the same **AppUser** domain entity and are distinguished through the **UserRole** attribute. The implementation does not use separate persistent classes for venue managers or PR managers.

Main services:

- validate user credentials;
- retrieve user and role information;
- retrieve and update user profiles;
- manage music preferences;
- manage profile privacy settings;
- upload and remove profile avatars;
- save and unsave events;
- retrieve saved events;
- manage demo payment methods;
- create support requests.

The main implementation elements are `AuthController`, `UserController`, `PaymentMethodController`, `SupportRequestController`, `AuthenticationService`, `UserService`, and the related data mediators and repositories.

User registration is not currently implemented. Authentication is limited to credential verification. After login, the frontend stores a session marker in browser `localStorage`; the backend does not currently create a server-side session or authentication token.

2.2.2 Event Discovery Component

This component provides event browsing, event-detail retrieval, searching, filtering, sorting, and popularity-based presentation.

Main services:

- retrieve available events;
- retrieve complete event details;
- search events using text;
- filter events by city, area, music genre, venue category, date, price, entry condition, and featured status;
- sort events by date, price, or stored popularity score;
- retrieve return-transport information associated with an event;
- retrieve partner venue information;
- include ticket availability, promotions, pregames, and social information in event DTOs.

The main implementation elements are `EventController`, `EventService`, `EventDataMediator`, `EventRepository`, and `NightOutMapper`.

Filtering is implemented through a sequence of conditional operations applied by `EventService` after retrieving the events. The current implementation does not use separate `FilterStrategy` classes.

Event popularity is represented by a score stored in the **Event** entity. The component can sort or display events using this value, but it does not currently calculate popularity through interchangeable strategies.

2.2.3 Venue and Event Administration Component

This component manages the venues and events associated with venue-manager accounts.

Main services:

- retrieve venues assigned to a venue manager;
- update venue information;
- create events;
- retrieve events created by a manager;
- update events;
- delete events;
- define and update event capacity;
- retrieve tickets associated with manager events;
- retrieve manager dashboard information and event statistics.

The main implementation elements are **ManagerController**, **VenueManagementService**, **DashboardService**, **VenueManagementDataMediator**, **DashboardDataMediator**, **VenueRepository**, and **EventRepository**.

Events created through this component are stored in the same database used by the Event Discovery Component. Consequently, newly created events become available to normal users through the event feed and event-detail endpoints.

The current prototype identifies the acting manager through an identifier supplied by the frontend. Some operations validate the manager role and event creator, but event creation and update do not yet consistently verify that the selected venue belongs to the supplied manager.

2.2.4 Ticketing Component

This component manages the complete ticket lifecycle for event attendance.

The current implementation uses the domain concept **Ticket** instead of **Reservation**.

Main services:

- request a ticket;
- create a ticket in the pending state;
- confirm a pending ticket when capacity is available;
- move a pending ticket to the waiting list when the event is full;
- prevent duplicate active tickets for the same user and event;
- retrieve user tickets;
- retrieve tickets associated with manager or PR accounts;

- cancel tickets;
- promote the oldest waiting-list ticket after a cancellation;
- expire unconfirmed tickets after the confirmation deadline;
- generate a digital QR payload for confirmed tickets;
- publish ticket-related notification events.

The main implementation elements are `TicketController`, `TicketService`, `TicketDataMediator`, `TicketRepository`, `Ticket`, `TicketStatus`, `TicketStateFactory`, and the concrete ticket-state classes.

Ticket lifecycle transitions are implemented using the State pattern. The persistent ticket stores a `TicketStatus`, while runtime state objects implement the operations allowed for each status.

Waiting-list promotion follows a first-in, first-out policy based on ticket creation time.

2.2.5 Pregame Component

This component manages pregame rooms associated with nightlife events.

Main services:

- create a pregame room;
- retrieve all pregame rooms;
- retrieve pregame rooms linked to a selected event;
- retrieve the details and participants of a room;
- join a room;
- leave a room;
- prevent duplicate room participation;
- prevent users from joining a full room;
- delete a room when requested by its host;
- create notifications for pregame operations.

The main implementation elements are `PregameController`, `PregameService`, `PregameDataMediator`, `PregameRoomRepository`, and `PregameRoom`.

Room membership and capacity rules are validated by the backend and persisted through JPA. Pregame notifications are currently created through direct calls from `PregameService` to `NotificationService`, rather than through Spring application events.

2.2.6 Friendship and Social Component

This component manages the social relationships between normal users and provides social information associated with events.

Main services:

- search for other users;

- send friendship requests;
- retrieve sent and received requests;
- accept or reject friendship requests;
- retrieve accepted friends;
- remove an existing friendship;
- retrieve friends attending a selected event;
- store saved-event participation records;
- provide social data used by the event feed and recommendation engine.

The main implementation elements are `FriendshipController`, `FriendshipService`, `FriendshipDataMediator`, `FriendshipRepository`, `Friendship`, `FriendshipStatus`, and `EventParticipation`.

The live social workflow is based on the `Friendship` entity. The older `SocialRelation` entity remains present in the repository and seed data, but it is not used by the current frontend or friendship services.

The current implementation does not provide a complete social activity feed. Event participation is mainly used to represent saved events, while confirmed event attendance is derived from confirmed tickets.

2.2.7 Notification Component

This component stores, retrieves, and updates notifications addressed to application users.

Main services:

- create a notification;
- retrieve notifications for a user;
- mark a notification as read;
- notify ticket requests and status changes;
- notify waiting-list promotion;
- notify friendship requests and accepted friendships;
- notify pregame room operations.

The main implementation elements are `NotificationController`, `NotificationService`, `NotificationDataMediator`, `UserNotificationRepository`, `UserNotification`, and the Spring application event listeners.

Ticket and friendship services publish Spring application events. Dedicated listeners react to these events and invoke `NotificationService`, providing an Observer-style decoupling between the originating component and notification persistence.

Pregame notifications currently use direct service calls. Runtime generation of friend-joined-event notifications and event-update notifications is not currently implemented, although corresponding notification types or seeded examples may exist.

2.2.8 Recommendation Component

This component generates personalized event recommendations for normal users.

Main services:

- build a recommendation context from user preferences, saved events, ticket history, social relations, location, and event data;
- exclude unsuitable or already attended events;
- evaluate candidate events using multiple recommendation strategies;
- combine the individual strategy contributions into a total score;
- rank recommended events;
- return recommendation reasons to the frontend.

The main implementation elements are `RecommendationController`, `RecommendationService`, `RecommendationDataMediator`, `RecommendationEngine`, and the concrete recommendation strategies.

The implementation uses six Spring-managed strategies:

- music-preference strategy;
- friends-attending strategy;
- distance strategy;
- booking-history strategy;
- saved-events strategy;
- popularity-contribution strategy.

All applicable strategies can contribute to the score of the same event. Therefore, the engine combines multiple strategy results instead of selecting only one algorithm for each recommendation request.

2.2.9 Promotion and PR Component

This component manages event promotions and PR-specific application functionality.

Main services:

- retrieve promotions associated with managed venues;
- create promotions;
- update promotion information;
- activate or deactivate promotions;
- delete promotions;
- associate promotions with venues and events;
- retrieve PR dashboard information;
- retrieve events assigned to a PR manager;
- retrieve tickets attributed to a PR manager;
- calculate or display ticket commission information.

The main implementation elements are `PromotionController`, `PrController`, `PromotionService`, `PrDashboardService`, the related data mediators, `Promotion`, `PrEventAssignment`, and the corresponding repositories.

PR managers are represented as `AppUser` instances with the `PR_MANAGER` role. PR functionality is an implemented extension that was not fully represented in the initial Design Document.

2.2.10 Data Access and Persistence Component

This component provides access to persistent application data.

The implementation uses multiple focused data mediators instead of the single global `NightOutDataMediator` proposed in the initial design. Each mediator supports a specific functional area and exposes the repository operations required by the corresponding application services.

Main responsibilities:

- retrieve and persist users;
- retrieve and persist venues and events;
- retrieve and persist tickets;
- retrieve and persist pregame rooms and participants;
- retrieve and persist friendships and event participation;
- retrieve and persist notifications;
- retrieve and persist promotions and PR assignments;
- coordinate multi-repository recommendation queries;
- coordinate manager dashboard queries;
- calculate repository-based counts and aggregate information.

The main implementation elements include the focused mediator classes, the Spring Data JPA repositories, the JPA domain entities, and the H2 database.

The normal data-access path is:

`Application Service` → `Focused Data Mediator` → `Spring Data JPA Repository` → `H2 Database`.

Two limited exceptions exist in the current implementation. `AuthenticationService` accesses `AppUserRepository` directly, while `NightOutMapper` performs additional repository queries when constructing aggregate event DTOs.

The H2 database is configured as an in-memory database using Hibernate `create-drop`. Data remains available during the current backend execution but is recreated when the backend restarts.

Avatar binary files are stored in the local file system, while the corresponding avatar URL is stored in the user entity.

2.3 Class View

The class view describes the main persistent entities, domain relationships, application-specific behavioral classes, and data-access coordination classes used by the current NightOUT implementation.

The implementation follows an object-oriented structure, although not all domain behavior is placed directly inside persistent entities. Most JPA entities represent persistent state and relationships, while application services implement use-case orchestration and business rules.

The main exception is the ticket lifecycle, where the `Ticket` entity validates status transitions and collaborates with runtime State objects selected through `TicketStateFactory`.

The current implementation also uses:

- Spring application events and listeners for ticket and friendship notifications;
- multiple recommendation strategies registered as Spring components;
- multiple focused data mediators instead of one global repository mediator.

To preserve diagram readability, the class view is divided into several diagrams. Only attributes and operations relevant to the software architecture are shown.

2.3.1 Core Domain Class Diagram

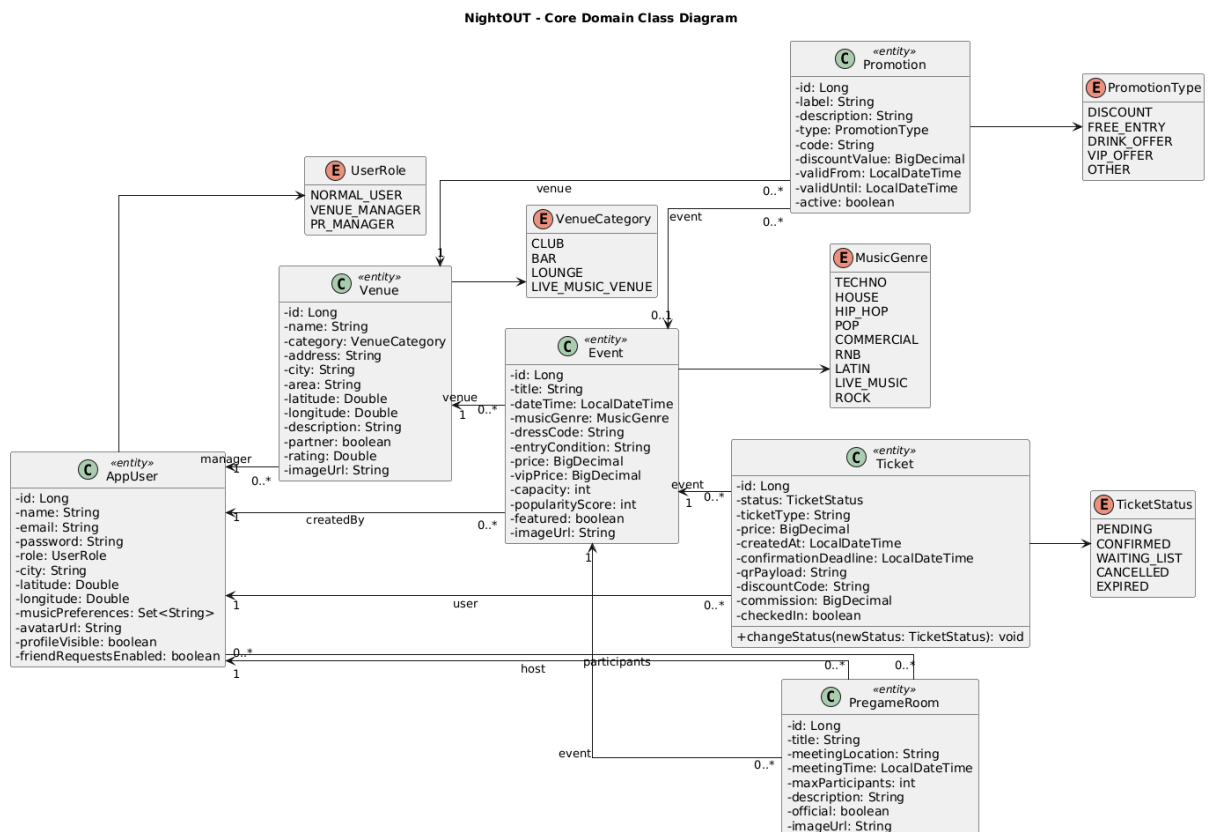
The core domain model is centred on users, venues, events, tickets, pregame rooms, and promotions.

All application accounts are represented by the `AppUser` entity. Normal users, venue managers, and PR managers are distinguished through the `UserRole` attribute rather than through separate subclasses.

A venue is assigned to a manager and may host multiple events. Each event is associated with one venue and records the user who created it.

The `Ticket` entity represents a user's request to attend an event. It replaces the `Reservation` entity proposed in the initial design and stores the persistent ticket status, ticket type, price, confirmation deadline, QR payload, and other ticket-related information.

Pregame rooms are linked to events, have one host, and maintain a many-to-many set of participants. Promotions may be associated with venues and, optionally, with individual events.



@startuml

title NightOUT - Core Domain Class Diagram

left to right direction

skinparam classAttributeIconSize 0

skinparam shadowing false

skinparam linetype ortho

hide empty methods

```

enum UserRole {
    NORMAL_USER
    VENUE_MANAGER
    PR_MANAGER
}
  
```

```

enum MusicGenre {
    TECHNO
    HOUSE
    HIP_HOP
    POP
    COMMERCIAL
    RNB
    LATIN
    LIVE_MUSIC
    ROCK
}
  
```

```
}

enum VenueCategory {
    CLUB
    BAR
    LOUNGE
    LIVE_MUSIC_VENUE
}

enum TicketStatus {
    PENDING
    CONFIRMED
    WAITING_LIST
    CANCELLED
    EXPIRED
}

enum PromotionType {
    DISCOUNT
    FREE_ENTRY
    DRINK_OFFER
    VIP_OFFER
    OTHER
}

class AppUser <<entity>> {
    - id: Long
    - name: String
    - email: String
    - password: String
    - role: UserRole
    - city: String
    - latitude: Double
    - longitude: Double
    - musicPreferences: Set<String>
    - avatarUrl: String
    - profileVisible: boolean
    - friendRequestsEnabled: boolean
}

class Venue <<entity>> {
    - id: Long
    - name: String
    - category: VenueCategory
    - address: String
    - city: String
    - area: String
    - latitude: Double
    - longitude: Double
    - description: String
    - partner: boolean
    - rating: Double
    - imageUrl: String
}
```

```
class Event <<entity>> {
  - id: Long
  - title: String
  - dateTime: LocalDateTime
  - musicGenre: MusicGenre
  - dressCode: String
  - entryCondition: String
  - price: BigDecimal
  - vipPrice: BigDecimal
  - capacity: int
  - popularityScore: int
  - featured: boolean
  - imageUrl: String
}

class Ticket <<entity>> {
  - id: Long
  - status: TicketStatus
  - ticketType: String
  - price: BigDecimal
  - createdAt: LocalDateTime
  - confirmationDeadline: LocalDateTime
  - qrPayload: String
  - discountCode: String
  - commission: BigDecimal
  - checkedIn: boolean
  + changeStatus(newStatus: TicketStatus): void
}

class PregameRoom <<entity>> {
  - id: Long
  - title: String
  - meetingLocation: String
  - meetingTime: LocalDateTime
  - maxParticipants: int
  - description: String
  - official: boolean
  - imageUrl: String
}

class Promotion <<entity>> {
  - id: Long
  - label: String
  - description: String
  - type: PromotionType
  - code: String
  - discountValue: BigDecimal
  - validFrom: LocalDateTime
  - validUntil: LocalDateTime
  - active: boolean
}

AppUser --> UserRole
```

```

Event --> MusicGenre
Venue --> VenueCategory
Ticket --> TicketStatus
Promotion --> PromotionType

AppUser "1" <-- "0..*" Venue : manager

Venue "1" <-- "0..*" Event : venue
AppUser "1" <-- "0..*" Event : createdBy

AppUser "1" <-- "0..*" Ticket : user
Event "1" <-- "0..*" Ticket : event

Event "1" <-- "0..*" PregameRoom : event
AppUser "1" <-- "0..*" PregameRoom : host
PregameRoom "0..*" -- "0..*" AppUser : participants

Venue "1" <-- "0..*" Promotion : venue
Event "0..1" <-- "0..*" Promotion : event

note bottom of AppUser
Venue managers and PR managers
are roles of AppUser.
They are not separate entities.
end note

note bottom of Ticket
Ticket replaces the Reservation
concept from the initial design.
end note

note bottom of Event
Popularity is currently stored
as an event attribute rather than
calculated by a dedicated strategy.
end note

@enduml

```

2.3.2 Social and Supporting Domain Classes

The social domain is based primarily on the **Friendship** entity.

A friendship records a directed request from a sender to a receiver and progresses through the values defined by **FriendshipStatus**. Once accepted, it is used by the application to retrieve social connections and friends attending events.

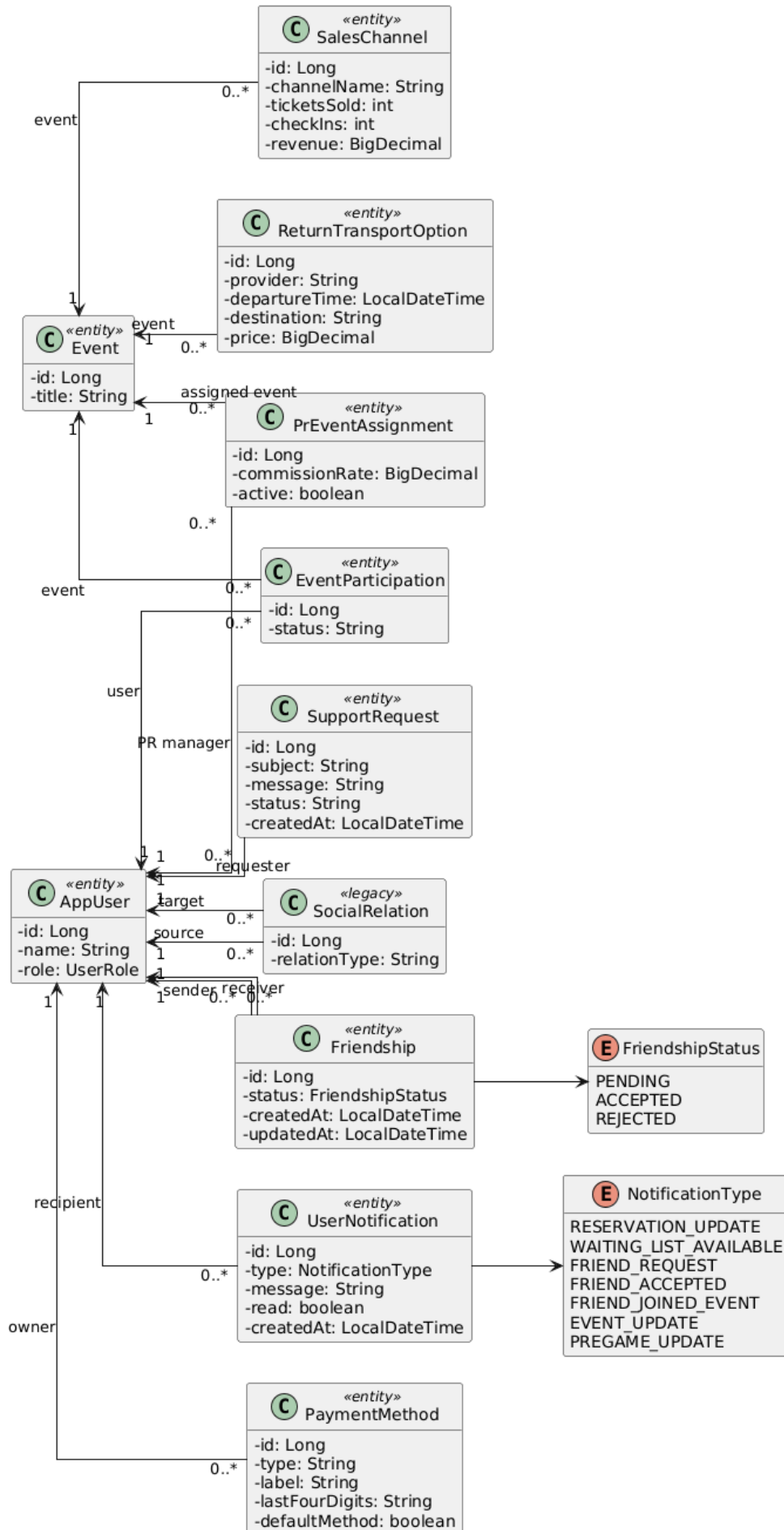
EventParticipation associates users with events. In the current implementation its status is stored as a string and is mainly used to represent saved events. Confirmed attendance is instead derived from confirmed tickets.

`UserNotification` stores notifications generated by ticket, friendship, and pregame operations.

The implementation also contains supporting entities for PR assignments, payment methods, support requests, return transport information, and sales-channel statistics.

A legacy `SocialRelation` entity is still present in the codebase and seed data, but it is not used by the current friendship workflow.

NightOUT - Social and Supporting Domain Classes



@startuml
title NightOUT - Social and Supporting Domain Classes

left to right direction

skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam linetype ortho
hide empty methods

```
enum FriendshipStatus {  
    PENDING  
    ACCEPTED  
    REJECTED  
}
```

```
enum NotificationType {  
    RESERVATION_UPDATE  
    WAITING_LIST_AVAILABLE  
    FRIEND_REQUEST  
    FRIEND_ACCEPTED  
    FRIEND_JOINED_EVENT  
    EVENT_UPDATE  
    PREGAME_UPDATE  
}
```

```
class AppUser <<entity>> {  
    - id: Long  
    - name: String  
    - role: UserRole  
}
```

```
class Event <<entity>> {  
    - id: Long  
    - title: String  
}
```

```
class Friendship <<entity>> {  
    - id: Long  
    - status: FriendshipStatus  
    - createdAt: LocalDateTime  
    - updatedAt: LocalDateTime  
}
```

```
class EventParticipation <<entity>> {  
    - id: Long  
    - status: String  
}
```

```
class UserNotification <<entity>> {  
    - id: Long  
    - type: NotificationType  
    - message: String  
    - read: boolean  
}
```

```

- createdAt: LocalDateTime
}

class PrEventAssignment <<entity>> {
- id: Long
- commissionRate: BigDecimal
- active: boolean
}

class PaymentMethod <<entity>> {
- id: Long
- type: String
- label: String
- lastFourDigits: String
- defaultMethod: boolean
}

class SupportRequest <<entity>> {
- id: Long
- subject: String
- message: String
- status: String
- createdAt: LocalDateTime
}

class ReturnTransportOption <<entity>> {
- id: Long
- provider: String
- departureTime: LocalDateTime
- destination: String
- price: BigDecimal
}

class SalesChannel <<entity>> {
- id: Long
- channelName: String
- ticketsSold: int
- checkIns: int
- revenue: BigDecimal
}

class SocialRelation <<legacy>> {
- id: Long
- relationType: String
}

Friendship --> FriendshipStatus
UserNotification --> NotificationType

AppUser "1" <-- "0..*" Friendship : sender
AppUser "1" <-- "0..*" Friendship : receiver

AppUser "1" <-- "0..*" EventParticipation : user
Event "1" <-- "0..*" EventParticipation : event

```

AppUser "1" <-- "0..*" UserNotification : recipient

AppUser "1" <-- "0..*" PrEventAssignment : PR manager

Event "1" <-- "0..*" PrEventAssignment : assigned event

AppUser "1" <-- "0..*" PaymentMethod : owner

AppUser "1" <-- "0..*" SupportRequest : requester

Event "1" <-- "0..*" ReturnTransportOption : event

Event "1" <-- "0..*" SalesChannel : event

AppUser "1" <-- "0..*" SocialRelation : source

AppUser "1" <-- "0..*" SocialRelation : target

note bottom of EventParticipation

The status is currently stored
as a String.

The live workflow mainly uses
the value SAVED.

end note

note bottom of Friendship

Friendship is the entity used
by the current social features.

end note

note bottom of SocialRelation

Legacy entity.

Seeded data may exist, but the
current friendship services and
frontend do not use this class.

end note

@enduml

2.3.3 Ticket State Pattern Class Diagram

The State pattern is used to implement the ticket lifecycle.

TicketStatus is persisted in the database as part of the **Ticket** entity. When a lifecycle operation must be executed, **TicketStateFactory** selects the concrete runtime State object corresponding to the current status.

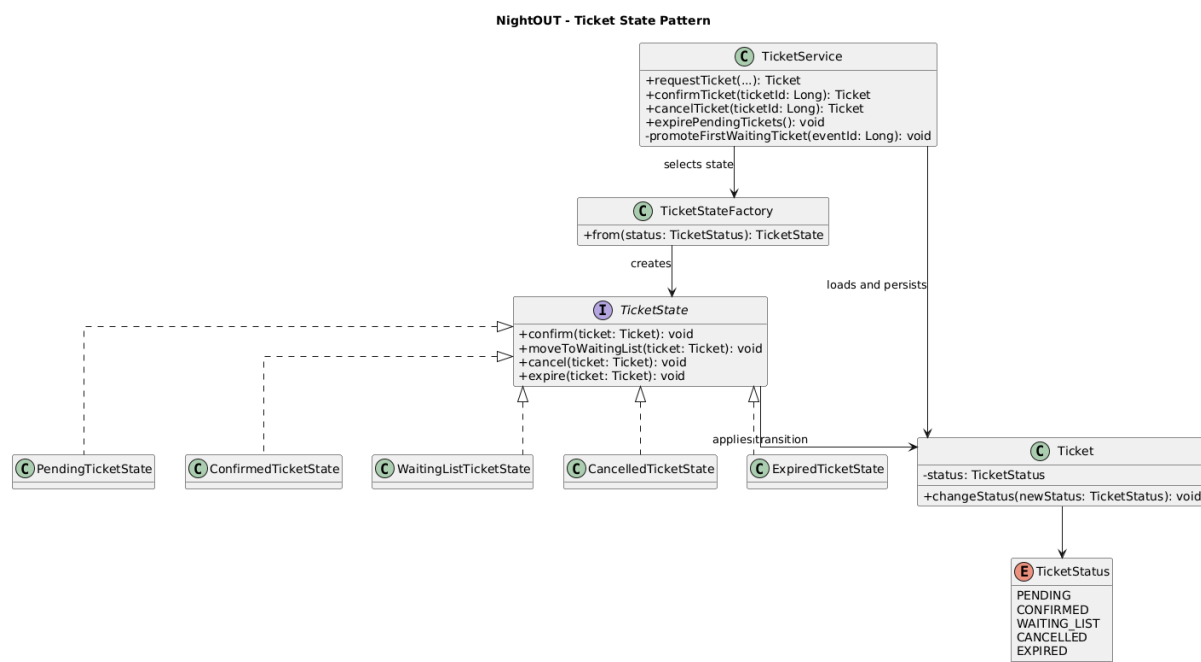
All ticket states implement the `TicketState` interface. Each state defines which operations are valid and which status transition must be applied.

The concrete states are:

- `PendingTicketState`;
- `ConfirmedTicketState`;
- `WaitingListTicketState`;
- `CancelledTicketState`;
- `ExpiredTicketState`.

`TicketService` coordinates ticket requests, confirmation, cancellation, expiration, capacity checks, and waiting-list promotion. It delegates the actual lifecycle operation to the selected State object.

The `Ticket` entity also validates the allowed transition matrix before changing the persistent `TicketStatus`. This provides an additional defensive validation layer.



@startuml
title NightOUT - Ticket State Pattern

top to bottom direction

skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam linetype ortho
hide empty fields

```

enum TicketStatus {
    PENDING
    CONFIRMED
    WAITING_LIST
    CANCELLED
    EXPIRED
}

interface TicketState {
    + confirm(ticket: Ticket): void
    + moveToWaitingList(ticket: Ticket): void
    + cancel(ticket: Ticket): void
    + expire(ticket: Ticket): void
}

class PendingTicketState
class ConfirmedTicketState
class WaitingListTicketState
class CancelledTicketState
class ExpiredTicketState

class TicketStateFactory {
    + from(status: TicketStatus): TicketState
}

class Ticket {
    - status: TicketStatus
    + changeStatus(newStatus: TicketStatus): void
}

class TicketService {
    + requestTicket(...): Ticket
    + confirmTicket(ticketId: Long): Ticket
    + cancelTicket(ticketId: Long): Ticket
    + expirePendingTickets(): void
    - promoteFirstWaitingTicket(eventId: Long): void
}

TicketState <|.. PendingTicketState
TicketState <|.. ConfirmedTicketState
TicketState <|.. WaitingListTicketState
TicketState <|.. CancelledTicketState
TicketState <|.. ExpiredTicketState

TicketService --> TicketStateFactory : selects state
TicketStateFactory --> TicketState : creates

TicketService --> Ticket : loads and persists
TicketState --> Ticket : applies transition
Ticket --> TicketStatus

note right of Ticket
TicketStatus is persisted

```

in the database.
end note

note right of TicketState
State objects are runtime
behavioral objects and are
not persisted.
end note

note bottom of TicketService
Capacity checks and FIFO waiting-list
selection are coordinated by the service
before or after the State transition.
end note

@enduml

2.3.4 Notification Event and Observer Class Diagram

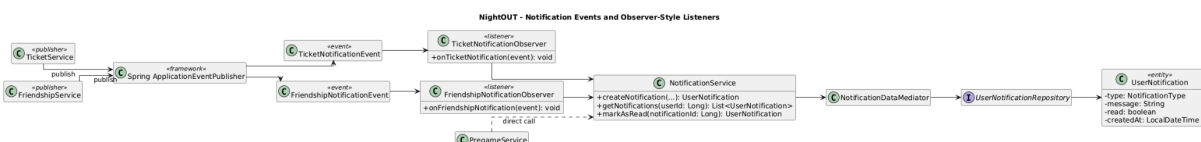
Notification decoupling is implemented through Spring application events rather than through manually managed Subject and Observer collections.

`TicketService` publishes ticket-related events, while `FriendshipService` publishes friendship-related events.

Spring automatically delivers these events to `TicketNotificationObserver` and `FriendshipNotificationObserver`, which are implemented as application-event listeners. The listeners invoke `NotificationService`, which creates and persists `UserNotification` entities.

This design provides Observer-style publish-subscribe behavior without requiring domain services to maintain explicit lists of registered observers.

Pregame notifications currently follow a different path: `PregameService` directly invokes `NotificationService`. Therefore, notification handling is currently a hybrid of event-driven and direct service communication.



@startuml

title NightOUT - Notification Events and Observer-Style Listeners

left to right direction

skinparam classAttributeIconSize 0

skinparam shadowing false

skinparam linetype ortho

hide empty fields

hide empty methods

class TicketService <<publisher>>

class FriendshipService <<publisher>>

class PregameService

class "Spring ApplicationEventPublisher" as EventPublisher <<framework>>

class TicketNotificationEvent <<event>>

class FriendshipNotificationEvent <<event>>

```
class TicketNotificationObserver <<listener>> {  
  + onTicketNotification(event): void  
}
```

```
class FriendshipNotificationObserver <<listener>> {  
  + onFriendshipNotification(event): void  
}
```

```
class NotificationService {  
  + createNotification(...): UserNotification  
  + getNotifications(userId: Long): List<UserNotification>  
  + markAsRead(notificationId: Long): UserNotification  
}
```

class NotificationDataMediator

interface UserNotificationRepository

```
class UserNotification <<entity>> {  
  - type: NotificationType  
  - message: String  
  - read: boolean  
  - createdAt: LocalDateTime  
}
```

TicketService --> EventPublisher : publish

FriendshipService --> EventPublisher : publish

EventPublisher --> TicketNotificationEvent

EventPublisher --> FriendshipNotificationEvent

TicketNotificationEvent --> TicketNotificationObserver : delivered by Spring

FriendshipNotificationEvent --> FriendshipNotificationObserver : delivered by Spring

TicketNotificationObserver --> NotificationService

FriendshipNotificationObserver --> NotificationService

PregameService ..> NotificationService : direct call

```
NotificationService --> NotificationDataMediator
NotificationDataMediator --> UserNotificationRepository
UserNotificationRepository --> UserNotification : persists
```

```
note bottom of EventPublisher
Listener registration and event delivery
are managed by the Spring container.
end note
```

```
note bottom of PregameService
Pregame notifications currently
do not use application events.
end note
```

```
note bottom of NotificationService
Runtime producers currently cover
ticket, waiting-list, friendship,
and pregame notifications.
```

```
Event-update and friend-attendance
producers are not implemented.
end note
```

```
@enduml
```

2.3.5 Recommendation Strategy Class Diagram

The Strategy pattern is used by the recommendation component.

RecommendationStrategy defines the common contract implemented by the different recommendation algorithms. Each concrete strategy evaluates one aspect of the relationship between a user and a candidate event.

The six implemented strategies are:

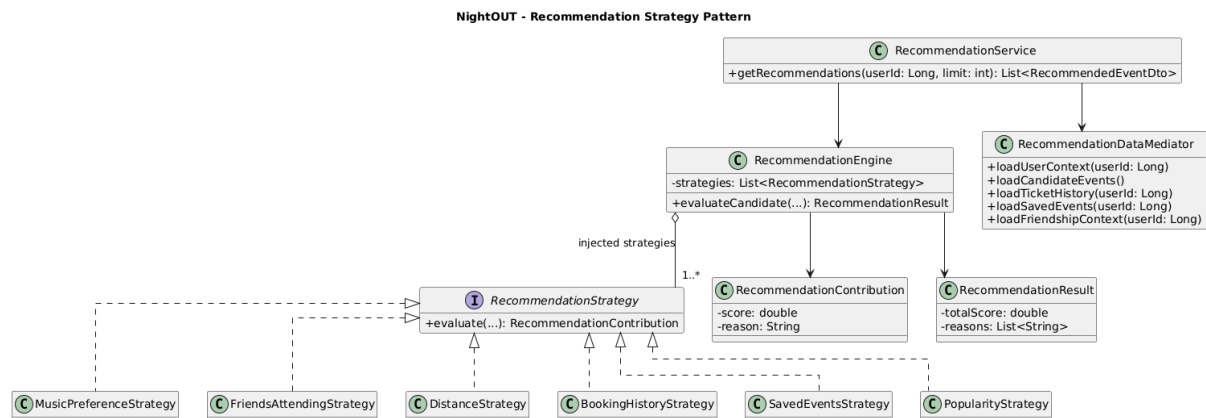
- **MusicPreferenceStrategy**;
- **FriendsAttendingStrategy**;
- **DistanceStrategy**;
- **BookingHistoryStrategy**;
- **SavedEventsStrategy**;
- **PopularityStrategy**.

The concrete strategies are registered as Spring components. **RecommendationEngine** receives the available strategy implementations through dependency injection.

For each candidate event, the engine evaluates all applicable strategies and combines their contributions into a total recommendation score. Therefore, strategies are not mutually exclusive: multiple strategies may contribute to the same recommendation.

RecommendationService retrieves the required user and event context through **RecommendationDataMediator**, invokes the engine, excludes unsuitable events, and returns the ranked results and recommendation reasons to the frontend.

The **PopularityStrategy** shown in this diagram contributes the stored event popularity value to the recommendation score. It does not calculate or update the event's popularity attribute.



@startuml
title NightOUT - Recommendation Strategy Pattern

top to bottom direction

skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam linetype ortho
hide empty fields

```
interface RecommendationStrategy {
    + evaluate(...): RecommendationContribution
}
```

```
class MusicPreferenceStrategy
class FriendsAttendingStrategy
class DistanceStrategy
class BookingHistoryStrategy
class SavedEventsStrategy
class PopularityStrategy
```

```
class RecommendationEngine {
    - strategies: List<RecommendationStrategy>
    + evaluateCandidate(...): RecommendationResult
}
```

```
class RecommendationService {
    + getRecommendations(userId: Long, limit: int): List<RecommendedEventDto>
}
```

```

class RecommendationDataMediator {
+ loadUserContext(userId: Long)
+ loadCandidateEvents()
+ loadTicketHistory(userId: Long)
+ loadSavedEvents(userId: Long)
+ loadFriendshipContext(userId: Long)
}

```

```

class RecommendationContribution {
- score: double
- reason: String
}

```

```

class RecommendationResult {
- totalScore: double
- reasons: List<String>
}

```

```

RecommendationStrategy <|.. MusicPreferenceStrategy
RecommendationStrategy <|.. FriendsAttendingStrategy
RecommendationStrategy <|.. DistanceStrategy
RecommendationStrategy <|.. BookingHistoryStrategy
RecommendationStrategy <|.. SavedEventsStrategy
RecommendationStrategy <|.. PopularityStrategy

```

```

RecommendationEngine o-- "1..*" RecommendationStrategy : injected strategies
RecommendationEngine --> RecommendationContribution
RecommendationEngine --> RecommendationResult

```

```

RecommendationService --> RecommendationDataMediator
RecommendationService --> RecommendationEngine

```

note right of RecommendationEngine
All applicable strategy scores
are combined for each candidate.

The engine does not select only
one strategy.
end note

note bottom of PopularityStrategy
Uses the event's stored popularity
as one recommendation contribution.

It does not compute the persistent
popularity score.
end note

@enduml

2.3.6 Focused Data Mediators and Repository Coordination

Repository coordination is implemented through multiple focused data mediators.

Each application service normally depends on a mediator associated with its functional area. The mediator exposes the required persistence operations and delegates them to one or more Spring Data JPA repositories.

Examples include:

- `UserDataMediator`;
- `EventDataMediator`;
- `TicketDataMediator`;
- `PregameDataMediator`;
- `FriendshipDataMediator`;
- `NotificationDataMediator`;
- `VenueManagementDataMediator`;
- `RecommendationDataMediator`;
- `DashboardDataMediator`.

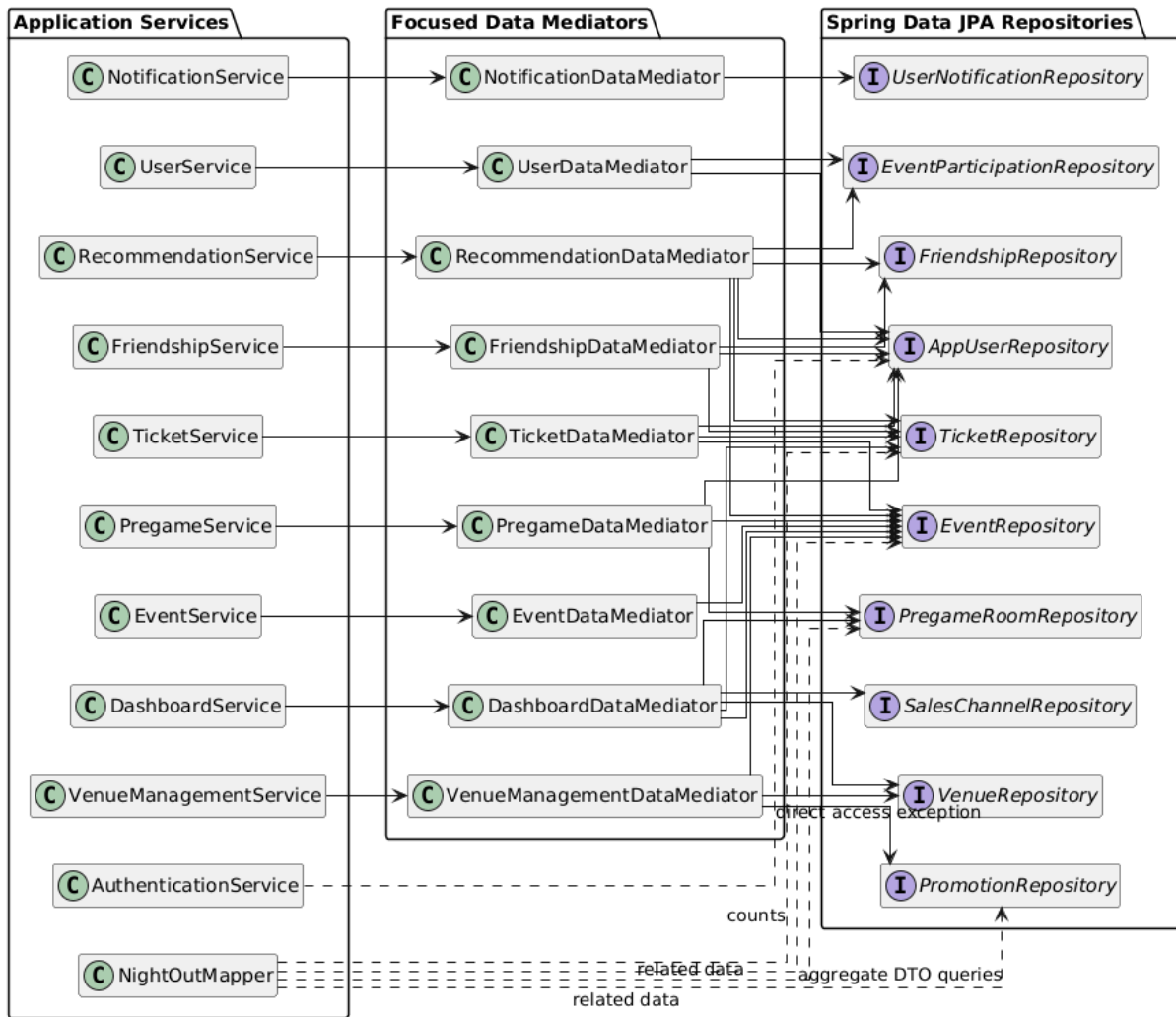
Some mediators act mainly as focused repository façades, while `RecommendationDataMediator` and `DashboardDataMediator` coordinate data from several repositories.

The implementation does not define a common mediator superclass or a single global `NightOutDataMediator`.

Two limited exceptions to this dependency direction exist:

- `AuthenticationService` accesses `AppUserRepository` directly;
- `NightOutMapper` accesses repositories while constructing aggregate event DTOs.

NightOUT - Focused Data Mediators and Repository Coordination



@startuml

title NightOUT - Focused Data Mediators and Repository Coordination

left to right direction

skinparam classAttributeIconSize 0

skinparam shadowing false

skinparam linetype ortho

hide empty fields

hide empty methods

package "Application Services" {

class UserService

class EventService

class TicketService

class PregameService

class FriendshipService

class NotificationService

```

class VenueManagementService
class RecommendationService
class DashboardService
class AuthenticationService
class NightOutMapper
}

package "Focused Data Mediators" {
class UserDataMediator
class EventDataMediator
class TicketDataMediator
class PregameDataMediator
class FriendshipDataMediator
class NotificationDataMediator
class VenueManagementDataMediator
class RecommendationDataMediator
class DashboardDataMediator
}

package "Spring Data JPA Repositories" {
interface AppUserRepository
interface VenueRepository
interface EventRepository
interface TicketRepository
interface PregameRoomRepository
interface FriendshipRepository
interface EventParticipationRepository
interface UserNotificationRepository
interface PromotionRepository
interface SalesChannelRepository
}

UserService --> UserDataMediator
EventService --> EventDataMediator
TicketService --> TicketDataMediator
PregameService --> PregameDataMediator
FriendshipService --> FriendshipDataMediator
NotificationService --> NotificationDataMediator
VenueManagementService --> VenueManagementDataMediator
RecommendationService --> RecommendationDataMediator
DashboardService --> DashboardDataMediator

UserDataMediator --> AppUserRepository
UserDataMediator --> EventParticipationRepository

EventDataMediator --> EventRepository

TicketDataMediator --> TicketRepository
TicketDataMediator --> EventRepository
TicketDataMediator --> AppUserRepository

PregameDataMediator --> PregameRoomRepository
PregameDataMediator --> EventRepository
PregameDataMediator --> AppUserRepository

```

FriendshipDataMediator --> FriendshipRepository
FriendshipDataMediator --> AppUserRepository
FriendshipDataMediator --> TicketRepository

NotificationDataMediator --> UserNotificationRepository

VenueManagementDataMediator --> VenueRepository
VenueManagementDataMediator --> EventRepository
VenueManagementDataMediator --> PromotionRepository

RecommendationDataMediator --> AppUserRepository
RecommendationDataMediator --> EventRepository
RecommendationDataMediator --> TicketRepository
RecommendationDataMediator --> FriendshipRepository
RecommendationDataMediator --> EventParticipationRepository

DashboardDataMediator --> VenueRepository
DashboardDataMediator --> EventRepository
DashboardDataMediator --> TicketRepository
DashboardDataMediator --> PregameRoomRepository
DashboardDataMediator --> SalesChannelRepository

AuthenticationService ..> AppUserRepository : direct access exception
NightOutMapper ..> EventRepository : aggregate DTO queries
NightOutMapper ..> TicketRepository : counts
NightOutMapper ..> PregameRoomRepository : related data
NightOutMapper ..> PromotionRepository : related data

note bottom of RecommendationDataMediator
Coordinates several repositories
to build recommendation context.
end note

note bottom of DashboardDataMediator
Coordinates repository data used
by manager dashboard statistics.
end note

note bottom of AuthenticationService
Direct repository access is a limited
exception to the normal mediator path.
end note

note bottom of NightOutMapper
The mapper is not completely pure:
it performs additional repository reads
while constructing aggregate DTOs.
end note

@enduml

2.4 Runtime View

The runtime view describes how the main frontend and backend elements collaborate during the execution of the most relevant NightOUT use cases.

The sequence diagrams follow the actual implemented dependency path:

React Page → nightoutApi → REST Controller → Application Service → Focused Data Mediator → Spring Data Repository → H2 Database.

Depending on the use case, application services also collaborate with ticket State objects, recommendation strategies, Spring application events, notification listeners, and DTO mapping components.

The frontend and backend communicate using JSON request and response DTOs. Persistent state changes are stored in the shared H2 database and therefore become visible to all application views while the backend is running.

The current prototype does not use a server-side authenticated principal. Consequently, user, venue-manager, and PR-manager identifiers shown in the diagrams are supplied by the frontend as path parameters, query parameters, or request DTO fields.

2.4.1 User Searches and Filters Events

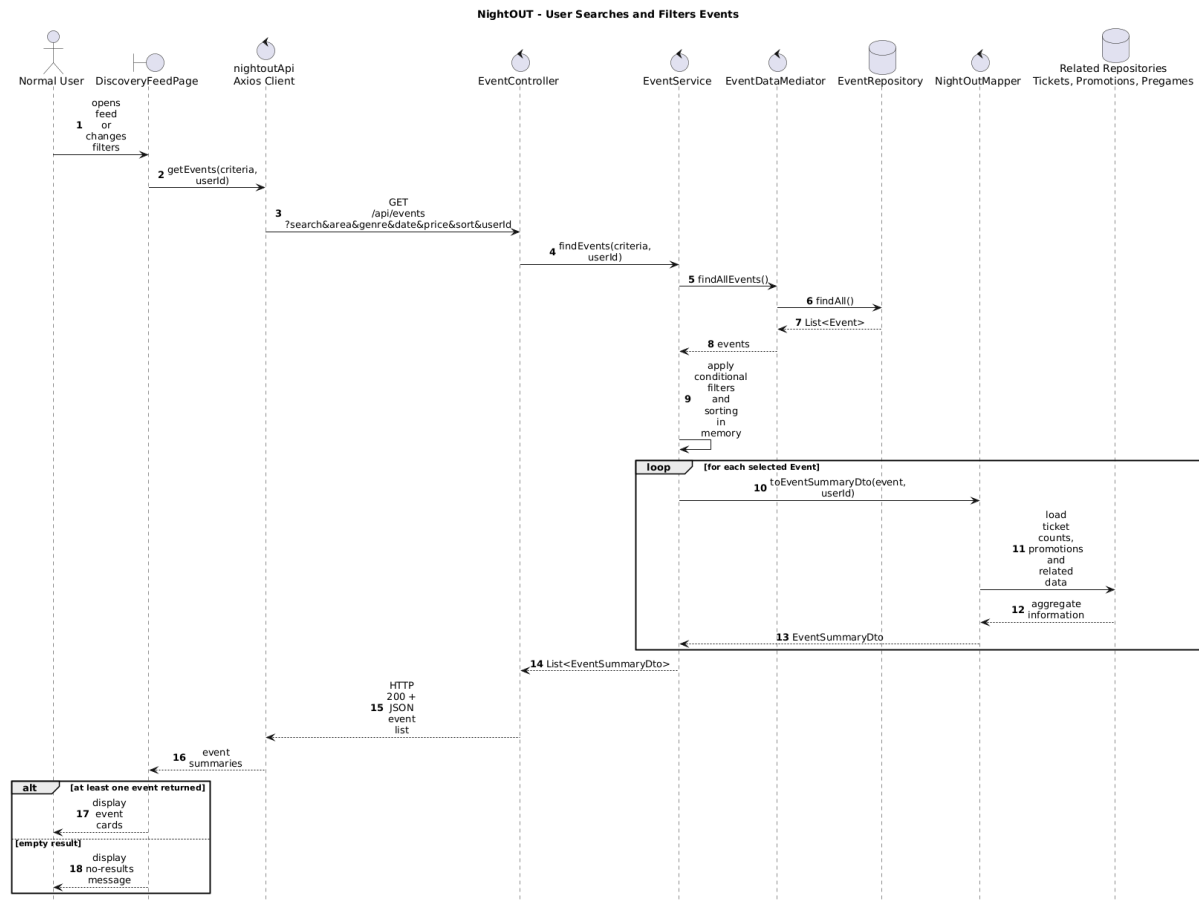
The event-discovery flow begins when a normal user opens the feed or modifies one of the available search and filter controls.

DiscoveryFeedPage passes the selected criteria to the shared Axios client, which sends a GET /api/events request to EventController.

EventService retrieves the available events through EventDataMediator and EventRepository. The filtering and sorting operations are then applied in memory through conditional predicates.

For each selected event, NightOutMapper constructs an EventSummaryDto. During this operation, the mapper performs additional repository queries to retrieve ticket counts, promotions, and other aggregate information.

The resulting event summaries are returned to the frontend and displayed as event cards. If the returned list is empty, the frontend displays the corresponding empty-result state.



2.4.2 User Requests and Confirms a Ticket

Ticket acquisition is implemented as a two-step process.

First, the user requests a ticket through `POST /api/tickets`. `TicketService` retrieves the selected user and event, verifies that no active ticket already exists for the same user and event, creates a new `Ticket` with status `PENDING`, and persists it.

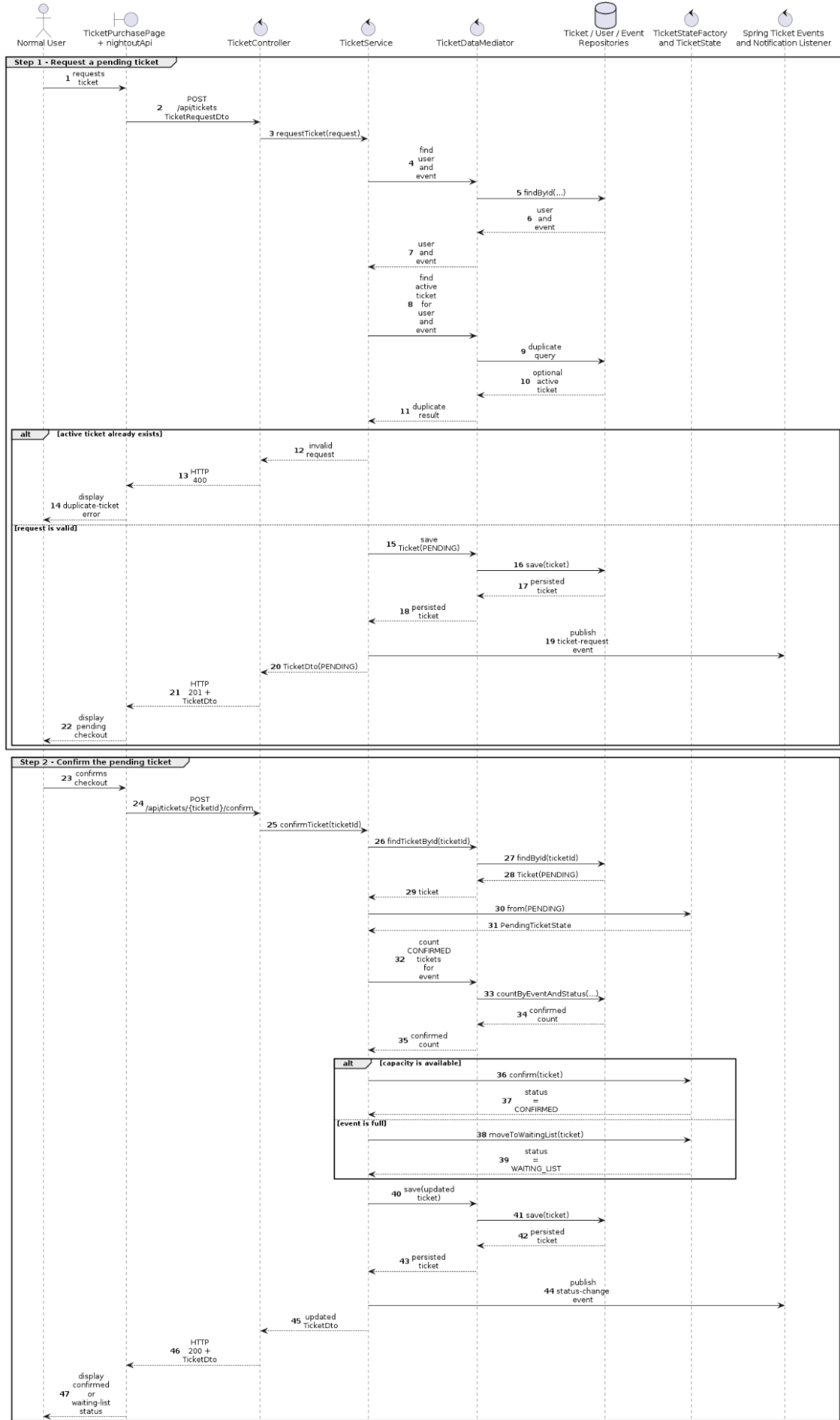
The frontend then performs the checkout confirmation through `POST /api/tickets/{ticketId}/confirm`.

During confirmation, `TicketService` retrieves the ticket, selects the corresponding runtime State object using `TicketStateFactory`, and counts the confirmed tickets associated with the event.

If capacity is available, `PendingTicketState` changes the ticket status to `CONFIRMED`. If the event is full, the same State object moves the ticket to `WAITING_LIST`.

The updated ticket is persisted and a ticket-related Spring application event is published. The resulting status is returned to the frontend and displayed to the user.

NightOUT - User Requests and Confirms a Ticket



```

@startuml
title NightOUT - User Requests and Confirms a Ticket

autonumber
hide footbox

skinparam shadowing false
skinparam sequenceMessageAlign center
skinparam maxMessageSize 45
skinparam ParticipantPadding 18
skinparam BoxPadding 10

actor "Normal User" as User
boundary "TicketPurchasePage\n+ nightoutApi" as Frontend
control "TicketController" as Controller
control "TicketService" as Service
control "TicketDataMediator" as Mediator
database "Ticket / User / Event\nRepositories" as Repositories
control "TicketStateFactory\nand TicketState" as State
control "Spring Ticket Events\nand Notification Listener" as Events

group Step 1 - Request a pending ticket

    User -> Frontend : requests ticket
    Frontend -> Controller : POST /api/tickets\nTicketRequestDto

    Controller -> Service : requestTicket(request)

    Service -> Mediator : find user and event
    Mediator -> Repositories : findById(...)
    Repositories --> Mediator : user and event
    Mediator --> Service : user and event

    Service -> Mediator : find active ticket\nfor user and event
    Mediator -> Repositories : duplicate query
    Repositories --> Mediator : optional active ticket
    Mediator --> Service : duplicate result

    alt active ticket already exists
        Service --> Controller : invalid request
        Controller --> Frontend : HTTP 400
        Frontend --> User : display duplicate-ticket error
    else request is valid
        Service -> Mediator : save Ticket(PENDING)
        Mediator -> Repositories : save(ticket)
        Repositories --> Mediator : persisted ticket
        Mediator --> Service : persisted ticket

        Service -> Events : publish ticket-request event

        Service --> Controller : TicketDto(PENDING)
        Controller --> Frontend : HTTP 201 + TicketDto
        Frontend --> User : display pending checkout
    end
end

```

end

end

group Step 2 - Confirm the pending ticket

User -> Frontend : confirms checkout

Frontend -> Controller : POST /api/tickets/{ticketId}/confirm

Controller -> Service : confirmTicket(ticketId)

Service -> Mediator : findTicketById(ticketId)

Mediator -> Repositories : findById(ticketId)

Repositories --> Mediator : Ticket(PENDING)

Mediator --> Service : ticket

Service -> State : from(PENDING)

State --> Service : PendingTicketState

Service -> Mediator : count CONFIRMED tickets\for event

Mediator -> Repositories : countByEventAndStatus(...)

Repositories --> Mediator : confirmed count

Mediator --> Service : confirmed count

alt capacity is available

Service -> State : confirm(ticket)

State --> Service : status = CONFIRMED

else event is full

Service -> State : moveToWaitingList(ticket)

State --> Service : status = WAITING_LIST

end

Service -> Mediator : save(updated ticket)

Mediator -> Repositories : save(ticket)

Repositories --> Mediator : persisted ticket

Mediator --> Service : persisted ticket

Service -> Events : publish status-change event

Service --> Controller : updated TicketDto

Controller --> Frontend : HTTP 200 + TicketDto

Frontend --> User : display confirmed or\nwaiting-list status

end

note over State

TicketStatus is persisted in Ticket.

Concrete TicketState objects provide
runtime transition behavior.

end note

note over Frontend

Payment data is part of the

prototype checkout interface.

No external payment provider
is integrated.
end note

@enduml

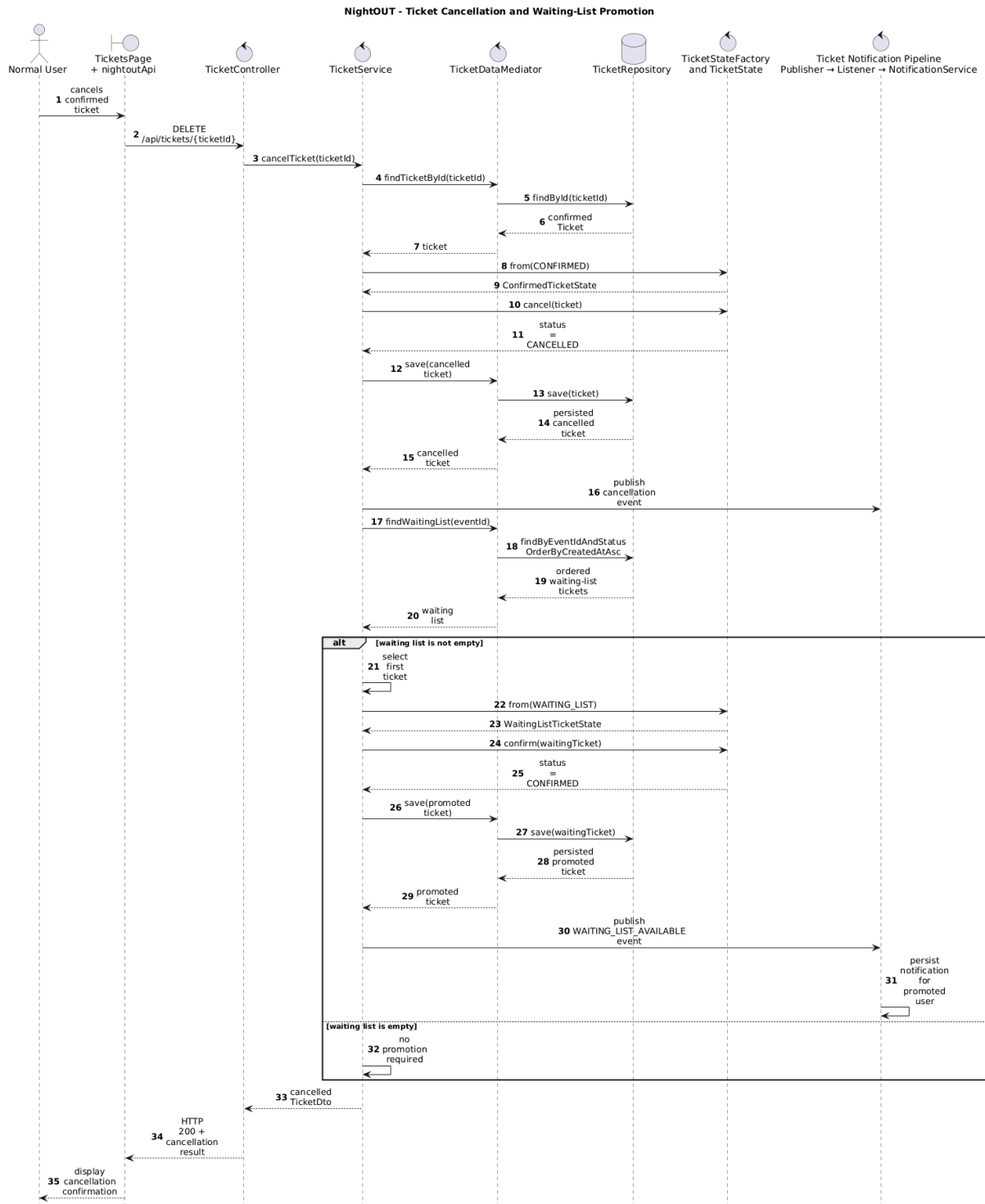
2.4.3 Ticket Cancellation and Waiting-List Promotion

When a confirmed ticket is cancelled, `TicketService` retrieves the ticket and delegates the cancellation transition to the runtime State object associated with its current status.

After persisting the cancelled ticket, the service searches for waiting-list tickets associated with the same event. The repository query orders the waiting list by ticket creation time.

If the waiting list is not empty, the oldest ticket is selected. `WaitingListTicketState` changes its status to `CONFIRMED`, and the promoted ticket is persisted.

Ticket cancellation and waiting-list promotion publish Spring application events. The ticket notification listener invokes `NotificationService`, which persists the corresponding user notifications.



@startuml

title NightOUT - Ticket Cancellation and Waiting-List Promotion

autonumber
hide footbox

skinparam shadowing false
skinparam sequenceMessageAlign center
skinparam maxMessageSize 45

skinparam ParticipantPadding 18
skinparam BoxPadding 10

actor "Normal User" as User
boundary "TicketsPage\n+ nightoutApi" as Frontend
control "TicketController" as Controller
control "TicketService" as Service
control "TicketDataMediator" as Mediator
database "TicketRepository" as Repository
control "TicketStateFactory\nand TicketState" as State
control "Ticket Notification Pipeline\nPublisher → Listener → NotificationService" as Notifications

User -> Frontend : cancels confirmed ticket
Frontend -> Controller : DELETE /api/tickets/{ticketId}

Controller -> Service : cancelTicket(ticketId)

Service -> Mediator : findTicketById(ticketId)
Mediator -> Repository : findById(ticketId)
Repository --> Mediator : confirmed Ticket
Mediator --> Service : ticket

Service -> State : from(CONFIRMED)
State --> Service : ConfirmedTicketState

Service -> State : cancel(ticket)
State --> Service : status = CANCELLED

Service -> Mediator : save(cancelled ticket)
Mediator -> Repository : save(ticket)
Repository --> Mediator : persisted cancelled ticket
Mediator --> Service : cancelled ticket

Service -> Notifications : publish cancellation event

Service -> Mediator : findWaitingList(eventId)
Mediator -> Repository : findByEventIdAndStatus\nOrderByCreatedAtAsc
Repository --> Mediator : ordered waiting-list tickets
Mediator --> Service : waiting list

alt waiting list is not empty

Service -> Service : select first ticket

Service -> State : from(WAITING_LIST)
State --> Service : WaitingListTicketState

Service -> State : confirm(waitingTicket)
State --> Service : status = CONFIRMED

Service -> Mediator : save(promoted ticket)
Mediator -> Repository : save(waitingTicket)
Repository --> Mediator : persisted promoted ticket

Mediator --> Service : promoted ticket

Service -> Notifications : publish WAITING_LIST_AVAILABLE event

Notifications -> Notifications : persist notification\nfor promoted user

else waiting list is empty

Service -> Service : no promotion required

end

Service --> Controller : cancelled TicketDto

Controller --> Frontend : HTTP 200 + cancellation result

Frontend --> User : display cancellation confirmation

note over Repository

Waiting-list tickets are ordered
by createdAt.

Promotion follows FIFO order.

end note

note over Notifications

The notification pipeline represents:

ApplicationEventPublisher

→ TicketNotificationObserver

→ NotificationService

→ UserNotificationRepository

end note

@enduml

2.4.4 User Joins or Leaves a Pregame Room

Pregame-room membership is managed through dedicated join and leave operations.

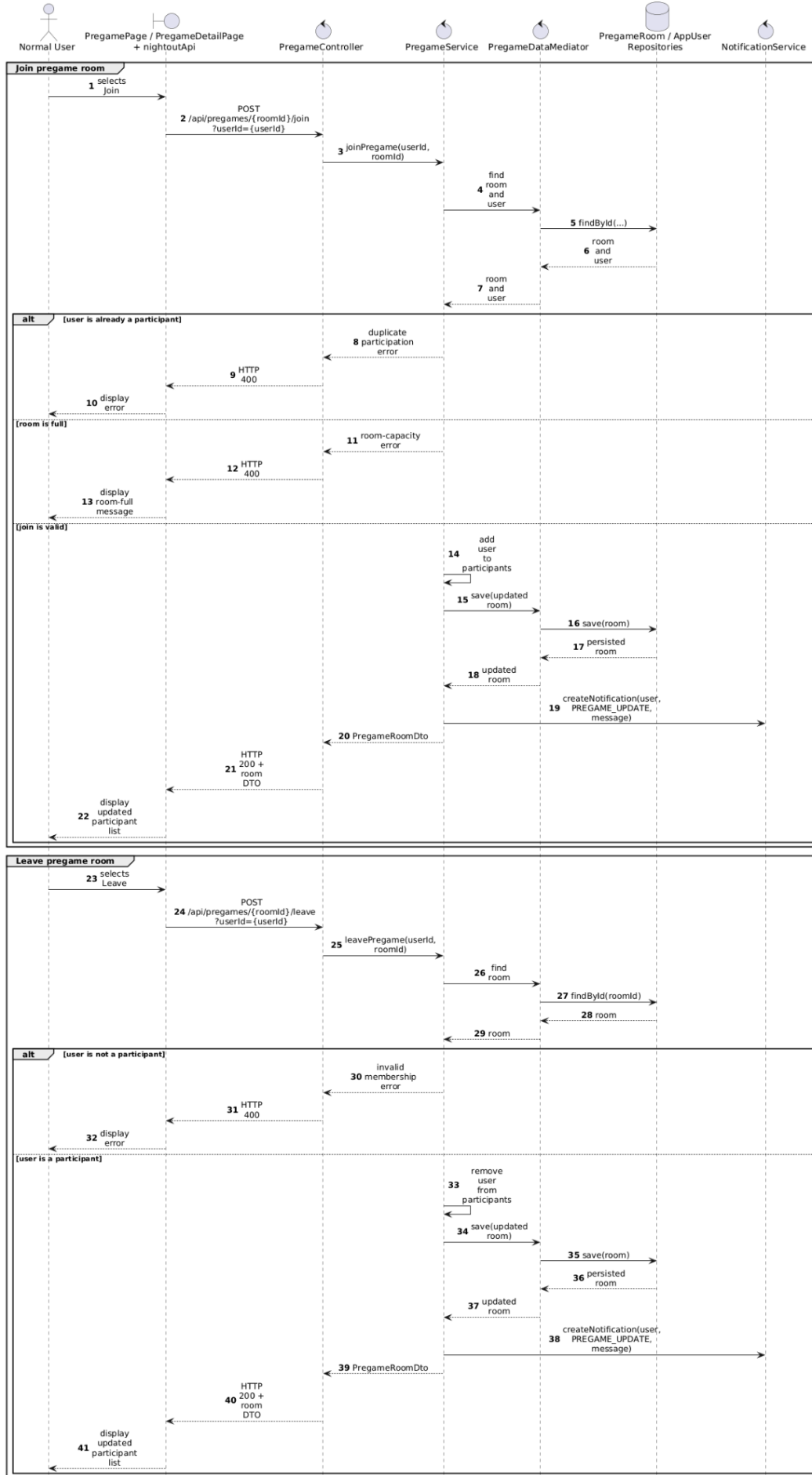
When a user joins a room, **PregameService** retrieves the room and the user through **PregameDataMediator**. The service verifies that the user is not already a participant and that the room has not reached its maximum capacity.

If both conditions are satisfied, the user is added to the room's participant collection and the updated room is persisted. **PregameService** then directly invokes **NotificationService**.

Leaving a room follows a similar flow. The service verifies that the user is currently a participant, removes the user, persists the room, and creates a notification.

Unlike ticket and friendship notifications, the current pregame flow does not use Spring application events.

NightOUT - User Joins or Leaves a Pregame Room



```

@startuml
title NightOUT - User Joins or Leaves a Pregame Room

autonumber
hide footbox

skinparam shadowing false
skinparam sequenceMessageAlign center
skinparam maxMessageSize 45
skinparam ParticipantPadding 18
skinparam BoxPadding 10

actor "Normal User" as User
boundary "PregamePage / PregameDetailPage\n+ nightoutApi" as Frontend
control "PregameController" as Controller
control "PregameService" as Service
control "PregameDataMediator" as Mediator
database "PregameRoom / AppUser\nRepositories" as Repositories
control "NotificationService" as NotificationService

group Join pregame room

    User -> Frontend : selects Join
    Frontend -> Controller : POST /api/pregames/{roomId}/join\n?userId={userId}

    Controller -> Service : joinPregame(userId, roomId)

    Service -> Mediator : find room and user
    Mediator -> Repositories : findById(...)
    Repositories --> Mediator : room and user
    Mediator --> Service : room and user

    alt user is already a participant
        Service --> Controller : duplicate participation error
        Controller --> Frontend : HTTP 400
        Frontend --> User : display error
    else room is full
        Service --> Controller : room-capacity error
        Controller --> Frontend : HTTP 400
        Frontend --> User : display room-full message
    else join is valid
        Service -> Service : add user to participants

        Service -> Mediator : save(updated room)
        Mediator -> Repositories : save(room)
        Repositories --> Mediator : persisted room
        Mediator --> Service : updated room

        Service -> NotificationService : createNotification(user,\nPREGAME_UPDATE,
message)

        Service --> Controller : PregameRoomDto
        Controller --> Frontend : HTTP 200 + room DTO
    end
end

```

Frontend --> User : display updated participant list
end

end

group Leave pregame room

User -> Frontend : selects Leave

Frontend -> Controller : POST /api/pregames/{roomId}/leave\n?userId={userId}

Controller -> Service : leavePregame(userId, roomId)

Service -> Mediator : find room

Mediator -> Repositories : findById(roomId)

Repositories --> Mediator : room

Mediator --> Service : room

alt user is not a participant

Service --> Controller : invalid membership error

Controller --> Frontend : HTTP 400

Frontend --> User : display error

else user is a participant

Service -> Service : remove user from participants

Service -> Mediator : save(updated room)

Mediator -> Repositories : save(room)

Repositories --> Mediator : persisted room

Mediator --> Service : updated room

Service -> NotificationService : createNotification(user,\nPREGAME_UPDATE,
message)

Service --> Controller : PregameRoomDto

Controller --> Frontend : HTTP 200 + room DTO

Frontend --> User : display updated participant list

end

end

note over Service, NotificationService

Pregame notifications use a direct
service call rather than the Spring
application-event mechanism.

end note

note over Service

The current implementation permits
the host to leave the participants set
while remaining the recorded room host.

This edge case is not yet resolved.

end note

@enduml

2.4.5 Venue Manager Creates an Event

A venue manager creates an event through `ManagerCreateEventPage`.

The frontend sends a `CreateEventDto` to `POST /api/manager/events`.

`VenueManagementService` retrieves the supplied manager and venue, verifies that the supplied user has an allowed manager role, validates the event data, creates the new `Event`, and persists it through `VenueManagementDataMediator`.

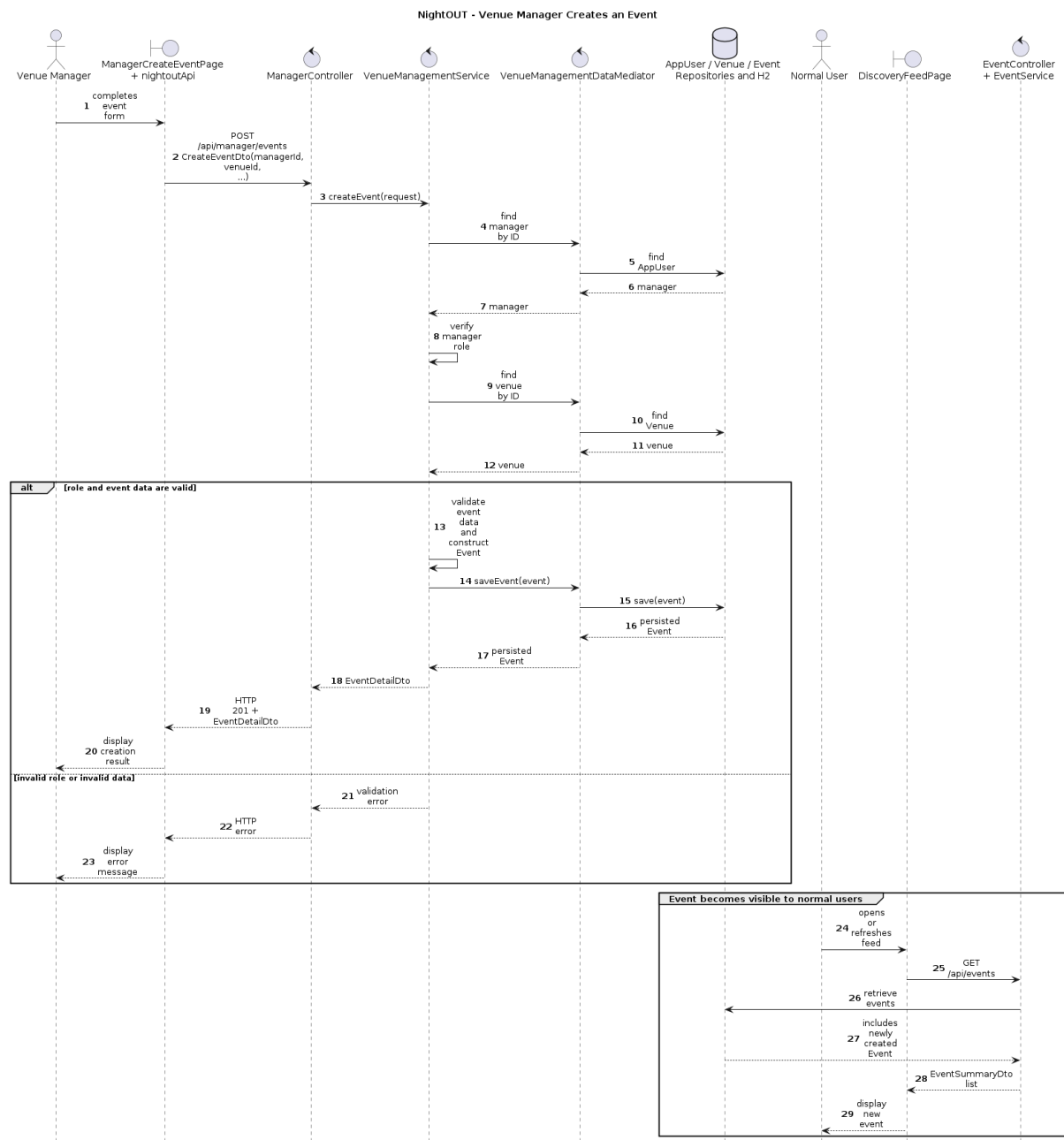
The event is stored in the same `EventRepository` used by `EventService`.

Consequently, a normal user can retrieve the newly created event through the global event feed or event-detail endpoint.

The current implementation does not verify that the selected venue is assigned to the supplied manager. Therefore, a manager identifier and a venue belonging to a different manager may currently be combined in a valid request.

After successful creation, the frontend currently attempts to navigate to the normal-user event-detail route. Since this route accepts only normal users in the frontend role guard, the

venue manager is redirected away from the newly created event.



@startuml
title NightOUT - Venue Manager Creates an Event

autonumber
hide footbox

skinparam shadowing false
skinparam sequenceMessageAlign center
skinparam maxMessageSize 45
skinparam ParticipantPadding 18

skinparam BoxPadding 10

actor "Venue Manager" as Manager
boundary "ManagerCreateEventPage\n+ nightoutApi" as ManagerFrontend
control "ManagerController" as ManagerController
control "VenueManagementService" as VenueService
control "VenueManagementDataMediator" as Mediator
database "AppUser / Venue / Event\nRepositories and H2" as Repositories

actor "Normal User" as User
boundary "DiscoveryFeedPage" as UserFrontend
control "EventController\n+ EventService" as EventDiscovery

Manager -> ManagerFrontend : completes event form
ManagerFrontend -> ManagerController : POST
/api/manager/events\nCreateEventDto(managerId, venueId, ...)

ManagerController -> VenueService : createEvent(request)

VenueService -> Mediator : find manager by ID
Mediator -> Repositories : find AppUser
Repositories --> Mediator : manager
Mediator --> VenueService : manager

VenueService -> VenueService : verify manager role

VenueService -> Mediator : find venue by ID
Mediator -> Repositories : find Venue
Repositories --> Mediator : venue
Mediator --> VenueService : venue

note over VenueService
The current implementation checks
the manager role but does not verify
that venue.manager.id equals
request.managerId.
end note

alt role and event data are valid

VenueService -> VenueService : validate event data\nand construct Event

VenueService -> Mediator : saveEvent(event)
Mediator -> Repositories : save(event)
Repositories --> Mediator : persisted Event
Mediator --> VenueService : persisted Event

VenueService --> ManagerController : EventDetailDto
ManagerController --> ManagerFrontend : HTTP 201 + EventDetailDto
ManagerFrontend --> Manager : display creation result

else invalid role or invalid data

VenueService --> ManagerController : validation error

```
ManagerController --> ManagerFrontend : HTTP error
ManagerFrontend --> Manager : display error message
```

end

group Event becomes visible to normal users

```
User -> UserFrontend : opens or refreshes feed
UserFrontend -> EventDiscovery : GET /api/events
EventDiscovery -> Repositories : retrieve events
Repositories --> EventDiscovery : includes newly created Event
EventDiscovery --> UserFrontend : EventSummaryDto list
UserFrontend --> User : display new event
```

end

note over ManagerFrontend
After creation, the current page
navigates to /events/{id}.

That route is restricted to NORMAL_USER,
so a manager is redirected to /manager.
end note

note over Repositories
Manager administration and user discovery
share the same EventRepository and H2 data.
end note

@enduml

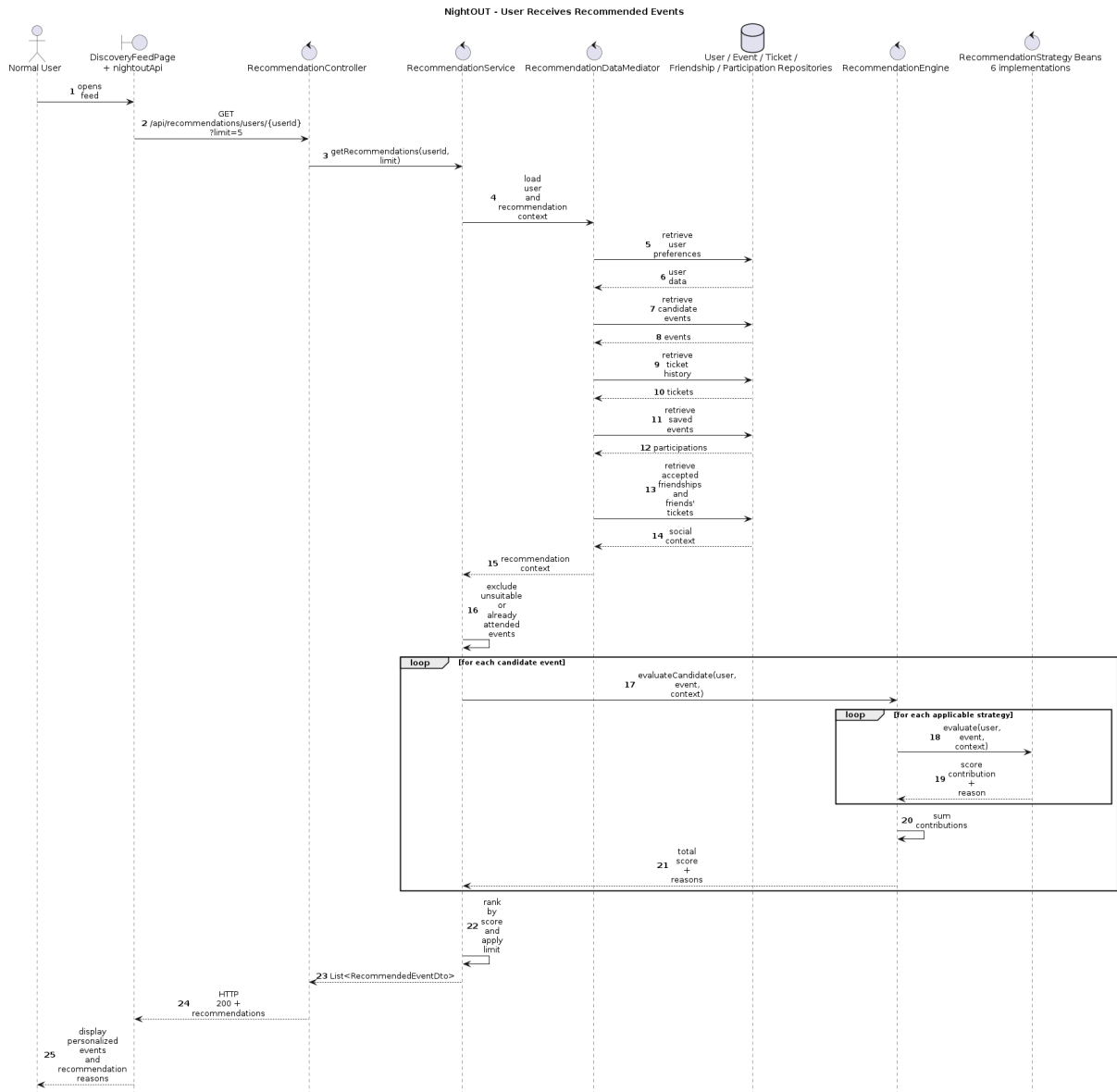
2.4.6 User Receives Recommended Events

Personalized recommendations are loaded when the normal-user feed requests recommended events.

`RecommendationService` uses `RecommendationDataMediator` to retrieve the user profile, candidate events, ticket history, saved events, and friendship context.

The resulting context is passed to `RecommendationEngine`. For each candidate event, the engine executes all applicable Spring-managed implementations of `RecommendationStrategy`.

The individual strategy contributions are combined into a total score and a list of recommendation reasons. Unsuitable or already attended events are excluded, and the remaining results are sorted and limited before being returned to the frontend.



@startuml

title NightOUT - User Receives Recommended Events

autonumber

hide footbox

skinparam shadowing false

skinparam sequenceMessageAlign center

skinparam maxMessageSize 45

skinparam ParticipantPadding 18

skinparam BoxPadding 10

actor "Normal User" as User

boundary "DiscoveryFeedPage\n+ nightoutApi" as Frontend

control "RecommendationController" as Controller

control "RecommendationService" as Service

control "RecommendationDataMediator" as Mediator

database "User / Event / Ticket / Friendship / Participation Repositories" as Repositories
control "RecommendationEngine" as Engine
control "RecommendationStrategy Beans" as Strategies

User -> Frontend : opens feed

Frontend -> Controller : GET /api/recommendations/users/{userId}?limit=5

Controller -> Service : getRecommendations(userId, limit)

Service -> Mediator : load user and recommendation context

Mediator -> Repositories : retrieve user preferences

Repositories --> Mediator : user data

Mediator -> Repositories : retrieve candidate events

Repositories --> Mediator : events

Mediator -> Repositories : retrieve ticket history

Repositories --> Mediator : tickets

Mediator -> Repositories : retrieve saved events

Repositories --> Mediator : participations

Mediator -> Repositories : retrieve accepted friendships and friends' tickets

Repositories --> Mediator : social context

Mediator --> Service : recommendation context

Service -> Service : exclude unsuitable or already attended events

loop for each candidate event

Service -> Engine : evaluateCandidate(user, event, context)

loop for each applicable strategy

Engine -> Strategies : evaluate(user, event, context)

Strategies --> Engine : score contribution + reason

end

Engine -> Engine : sum contributions

Engine --> Service : total score + reasons

end

Service -> Service : rank by score and apply limit

Service --> Controller : List<RecommendedEventDto>

Controller --> Frontend : HTTP 200 + recommendations

Frontend --> User : display personalized events and recommendation reasons

note over Strategies

Implemented strategies:

MusicPreference

```
FriendsAttending
Distance
BookingHistory
SavedEvents
Popularity
end note
```

```
note over Engine
Several strategies may contribute
to the same event.
```

```
The engine does not choose only
one strategy.
end note
```

```
@enduml
```

2.4.7 Venue Manager Loads the Dashboard

The manager dashboard retrieves aggregated information through `GET /api/manager/dashboard`.

In the current frontend implementation, `ManagerDashboardPage` invokes the dashboard API without passing the identifier of the logged-in venue manager.

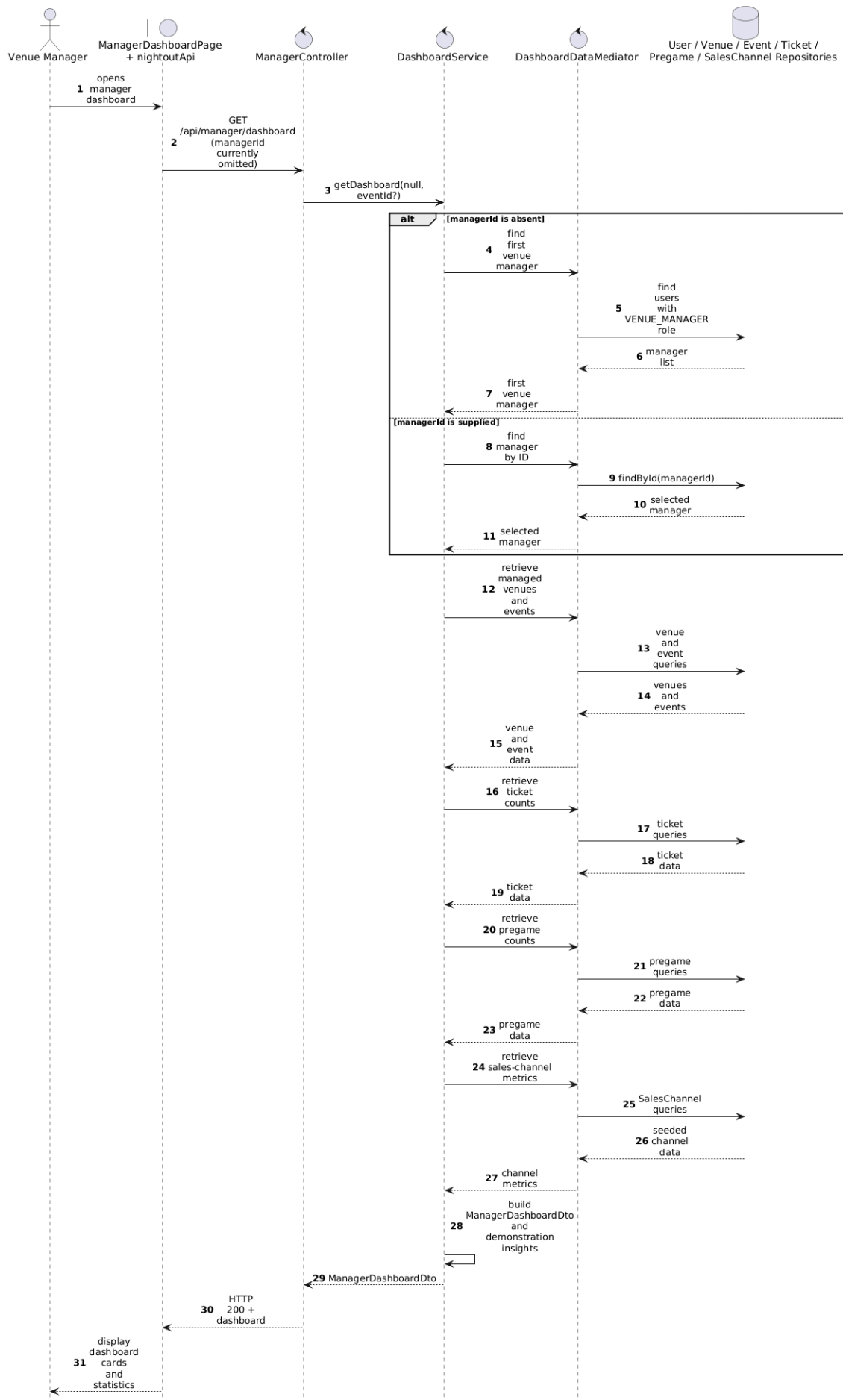
When `managerId` is absent, `DashboardService` selects the first venue manager returned by the persistence layer. It then uses `DashboardDataMediator` to retrieve the manager's venues, events, tickets, pregames, and sales-channel records.

The resulting dashboard DTO is returned to the frontend.

This behavior means that the dashboard is guaranteed to represent the correct manager only when the selected account corresponds to the first venue manager or when a manager identifier is explicitly supplied.

Some dashboard indicators also combine stored `SalesChannel` totals with counts derived from persisted `Ticket` entities. The two data sources currently represent different seeded populations. Some insight values are also hardcoded for demonstration purposes.

NightOUT - Venue Manager Loads the Dashboard



@startuml
title NightOUT - Venue Manager Loads the Dashboard

autonumber
hide footbox

skinparam shadowing false
skinparam sequenceMessageAlign center
skinparam maxMessageSize 45
skinparam ParticipantPadding 20
skinparam BoxPadding 10

actor "Venue Manager" as Manager
boundary "ManagerDashboardPage\n+ nightoutApi" as Frontend
control "ManagerController" as Controller
control "DashboardService" as Service
control "DashboardDataMediator" as Mediator
database "User / Venue / Event / Ticket /\nPregame / SalesChannel Repositories" as Repositories

Manager -> Frontend : opens manager dashboard

Frontend -> Controller : GET /api/manager/dashboard\n(managerId currently omitted)

Controller -> Service : getDashboard(null, eventId?)

alt managerId is absent

Service -> Mediator : find first venue manager
Mediator -> Repositories : find users with\nVENUE_MANAGER role
Repositories --> Mediator : manager list
Mediator --> Service : first venue manager

else managerId is supplied

Service -> Mediator : find manager by ID
Mediator -> Repositories : findById(managerId)
Repositories --> Mediator : selected manager
Mediator --> Service : selected manager

end

Service -> Mediator : retrieve managed venues and events
Mediator -> Repositories : venue and event queries
Repositories --> Mediator : venues and events
Mediator --> Service : venue and event data

Service -> Mediator : retrieve ticket counts
Mediator -> Repositories : ticket queries
Repositories --> Mediator : ticket data
Mediator --> Service : ticket data

Service -> Mediator : retrieve pregame counts
Mediator -> Repositories : pregame queries
Repositories --> Mediator : pregame data
Mediator --> Service : pregame data

Service -> Mediator : retrieve sales-channel metrics
Mediator -> Repositories : SalesChannel queries
Repositories --> Mediator : seeded channel data
Mediator --> Service : channel metrics

Service -> Service : build ManagerDashboardDto\nand demonstration insights

Service --> Controller : ManagerDashboardDto
Controller --> Frontend : HTTP 200 + dashboard
Frontend --> Manager : display dashboard cards\nand statistics

note over Frontend, Service
Because ManagerDashboardPage does not
currently pass the logged-in manager ID,
DashboardService resolves the first
venue manager by default.
end note

note over Service, Repositories
Dashboard metrics currently combine:

- persisted Ticket counts;
- seeded SalesChannel totals;
- some hardcoded insight values.

These sources do not always represent
the same statistical population.
end note

@enduml

2.5 Selected Architectural Styles and Patterns

The current NightOUT implementation combines architectural styles, object-oriented design patterns, and framework-supported mechanisms.

The main architectural choices are the client-server style and a predominantly layered backend structure. The most relevant behavioral design patterns are State and Strategy. Ticket-state selection also uses a Simple Factory, while notification decoupling is implemented through Spring application events and listeners, providing Observer-style publish-subscribe behavior.

Persistence access follows the Repository pattern through Spring Data JPA. Application services normally interact with repositories through focused data mediators, while Data Transfer Objects are used at the REST boundary to avoid exposing persistent entities directly.

Some patterns proposed in the initial design were implemented differently or were not required by the final prototype. In particular, event filtering does not use Strategy classes, event popularity is stored rather than calculated through a strategy, and repository coordination is distributed across focused mediators rather than centralized in one global mediator.

2.5.1 Client-Server Architecture

NightOUT uses a client-server architecture.

The client is a mobile-oriented React and TypeScript web application. It provides the graphical interface, page navigation, role-specific routing, local component state, and user interaction.

The server is a Java Spring Boot application. It exposes REST endpoints, validates request DTOs, executes application and domain logic, coordinates persistence, and returns response DTOs.

The client and server communicate through HTTP requests and JSON representations using the shared Axios-based `nightoutApi` client.

The main advantages of this architecture are:

- separation between presentation logic and backend application logic;
- independent development and testing of frontend and backend;
- centralized enforcement of ticket, pregame, friendship, and event-management rules;
- shared persistent data across normal-user, venue-manager, and PR-manager interfaces;
- the possibility of replacing or extending the frontend without directly modifying the domain and persistence layers.

The frontend does not contain the authoritative domain dataset. Events, tickets, pregames, friendships, promotions, and notifications are obtained from the backend.

The current prototype is designed primarily for local execution. The frontend communicates with a fixed local backend address, and authentication does not yet use server-side sessions or access tokens.

2.5.2 Layered Architecture

The Spring Boot backend follows a predominantly layered architecture.

The main layers are:

- **REST Controller Layer**, which receives HTTP requests, validates input DTOs, invokes application services, and returns response DTOs;

- **Application Service Layer**, which implements use-case orchestration, transaction boundaries, business rules, and coordination among domain and infrastructure components;
- **Domain and Behavioral Layer**, which contains JPA entities, enums, ticket State objects, recommendation strategies, and domain-related application events;
- **Data Mediation Layer**, which exposes focused persistence operations to application services;
- **Repository Layer**, which uses Spring Data JPA interfaces to retrieve and persist entities;
- **Persistence Layer**, which stores the prototype data in the H2 relational database.

The normal dependency direction is:

REST Controller → Application Service → Focused Data Mediator → Spring Data Repository → H2 Database.

This organization supports:

- separation of concerns;
- reduced coupling between HTTP handling and persistence;
- independent testing of application services;
- reuse of backend functionality across multiple frontend pages;
- clearer traceability between requirements, services, and persistent entities.

The implementation contains two limited deviations from the normal dependency direction:

- `AuthenticationService` accesses `AppUserRepository` directly;
- `NightOutMapper` performs additional repository queries while constructing aggregate event DTOs.

For this reason, the architecture is described as predominantly layered rather than strictly layered.

2.5.3 Repository and DTO-Based Communication

NightOUT uses the Repository pattern to separate persistence operations from application logic.

Spring Data JPA repository interfaces provide:

- entity retrieval by identifier;
- entity creation and update;
- deletion;
- relationship queries;
- ticket and participation counts;
- ordered waiting-list retrieval;
- friendship and event-assignment queries.

Application services normally do not depend directly on the H2 database or on SQL statements. They use focused data mediators, which delegate persistence operations to the appropriate repositories.

REST controllers also avoid exposing JPA entities directly. Requests and responses are represented through Data Transfer Objects.

The DTO boundary provides several advantages:

- persistent entity relationships are not serialized directly;
- internal fields, such as passwords, are not included in API responses;
- frontend representations can aggregate data from multiple entities;
- REST responses remain independent from internal JPA implementation details;
- bidirectional entity relationships do not produce recursive JSON serialization.

`NightOutMapper` is responsible for constructing many of the event-related DTOs. In the current implementation, the mapper also performs repository reads to include ticket counts, promotions, pregames, and other aggregate data. Although this produces complete DTOs, it means that the mapper is not a completely pure transformation component.

2.5.4 State Pattern

The State pattern is used to implement the lifecycle of a `Ticket`.

A ticket may have one of the following persistent statuses:

- `PENDING`;
- `CONFIRMED`;
- `WAITING_LIST`;
- `CANCELLED`;
- `EXPIRED`.

The operations that may be performed on a ticket depend on its current status. For example:

- a pending ticket may become confirmed when event capacity is available;
- a pending ticket may enter the waiting list when the event is full;
- a confirmed ticket may be cancelled;
- a waiting-list ticket may become confirmed when another confirmed ticket is cancelled;
- a cancelled or expired ticket cannot return to an active state.

All runtime states implement the `TicketState` interface. The concrete implementations are:

- `PendingTicketState`;
- `ConfirmedTicketState`;
- `WaitingListTicketState`;

- `CancelledTicketState;`
- `ExpiredTicketState.`

`TicketService` delegates lifecycle operations to the concrete State corresponding to the ticket's current status.

The persistent `Ticket` entity stores a `TicketStatus` rather than a reference to a State object. Runtime State instances are created only when a lifecycle operation is executed.

The entity also validates the permitted status-transition matrix before changing the persistent status. This provides an additional defensive validation layer.

The State pattern prevents lifecycle behavior from being concentrated in a large sequence of conditional statements inside `TicketService` and makes invalid transitions explicit.

2.5.5 Simple Factory for Ticket States

Ticket-state selection is implemented through a Simple Factory.

`TicketStateFactory` receives the persistent `TicketStatus` and returns the corresponding implementation of `TicketState`.

For example:

- `PENDING` produces `PendingTicketState;`
- `CONFIRMED` produces `ConfirmedTicketState;`
- `WAITING_LIST` produces `WaitingListTicketState;`
- `CANCELLED` produces `CancelledTicketState;`
- `EXPIRED` produces `ExpiredTicketState.`

This choice prevents `TicketService` from constructing concrete State objects directly throughout its methods.

The Factory centralizes the mapping between persistent statuses and runtime behavioral objects. It also separates State selection from the business operations that use the selected State.

The factory is a simple object-creation component rather than a persistent domain object. The generated State instances do not have their own database representation.

2.5.6 Observer-Style Publish-Subscribe

Notification decoupling is implemented through Spring application events and listeners, providing Observer-style publish-subscribe behavior.

`TicketService` publishes events associated with ticket requests, ticket-status changes, cancellations, expirations, and waiting-list promotion.

`FriendshipService` publishes events associated with friendship requests and accepted friendships.

Spring's `ApplicationEventPublisher` dispatches these events to registered listener components:

- `TicketNotificationObserver`;
- `FriendshipNotificationObserver`.

The listeners invoke `NotificationService`, which creates and persists the corresponding `UserNotification`.

The originating services therefore do not need to know how the notification is stored or displayed. They only publish a domain-related event.

The advantages of this solution are:

- reduced coupling between ticket or friendship operations and notification persistence;
- easier introduction of additional event listeners;
- centralized notification creation;
- use of Spring-managed listener registration rather than manually maintained observer lists.

The current implementation is hybrid rather than completely event-driven.

`PregameService` directly invokes `NotificationService` instead of publishing a Spring application event.

Runtime producers currently cover ticket, waiting-list, friendship-request, friendship-acceptance, and pregame notifications. Event-update notifications and friend-attendance notifications are not generated by the current runtime logic, even though related notification types or seed examples may exist.

2.5.7 Strategy Pattern for Recommendations

The Strategy pattern is used by the Recommendation Component.

`RecommendationStrategy` defines the common contract used to evaluate the relevance of a candidate event for a selected user.

The current implementation contains six Spring-managed strategies:

1. `MusicPreferenceStrategy`;
2. `FriendsAttendingStrategy`;
3. `DistanceStrategy`;
4. `BookingHistoryStrategy`;
5. `SavedEventsStrategy`;
6. `PopularityStrategy`.

`RecommendationEngine` receives the available implementations through Spring dependency injection.

For each candidate event, the engine evaluates every applicable strategy. Each strategy may produce:

- a score contribution;
- a reason explaining the recommendation.

The individual contributions are combined into a total recommendation score. The strategies are therefore additive and are not selected as mutually exclusive alternatives.

This design allows a new recommendation criterion to be introduced by adding another implementation of `RecommendationStrategy`, without changing the existing concrete strategies.

`RecommendationService` remains responsible for:

- retrieving the recommendation context;
- selecting candidate events;
- excluding unsuitable events;
- invoking the recommendation engine;
- ranking the results;
- limiting the number of returned recommendations.

The `PopularityStrategy` used in this component does not calculate the persistent popularity of an event. It only uses the stored popularity value as one contribution to the recommendation score.

2.5.8 Focused Data Mediation

Repository coordination is implemented through multiple focused data mediators.

The initial design proposed one global component named `NightOutDataMediator`. The implemented system instead distributes persistence coordination among components associated with specific functional areas.

Examples include:

- `UserDataMediator`;
- `EventDataMediator`;
- `TicketDataMediator`;
- `PregameDataMediator`;
- `FriendshipDataMediator`;
- `NotificationDataMediator`;
- `VenueManagementDataMediator`;
- `RecommendationDataMediator`;

- `DashboardDataMediator`.

Most application services depend on one focused mediator instead of directly depending on several repositories.

Some mediators primarily act as focused repository façades. Others coordinate multiple repositories and perform more substantial aggregation. In particular:

- `RecommendationDataMediator` retrieves user, event, ticket, saved-event, and friendship information;
- `DashboardDataMediator` retrieves venue, event, ticket, pregame, and sales-channel information.

This organization provides:

- smaller dependency sets for application services;
- domain-oriented persistence interfaces;
- reduced risk of creating one global data-access God Object;
- centralized multi-repository queries for complex application areas.

The implementation is described as mediator-like because the focused components coordinate access between services and repositories. However, it is not a textbook implementation of one central Mediator shared by all components.

`AuthenticationService` and `NightOutMapper` remain limited exceptions because they access repositories directly.

3. User Interface Design

3.1 UI Overview

NightOUT is implemented as a mobile-oriented React and TypeScript web application.

The user interface is organized around three application roles:

- normal user;
- venue manager;
- PR manager.

Each role is associated with a dedicated set of routes and pages. After login, the frontend redirects the user to the appropriate interface according to the role returned by the backend.

The main UI design principles are:

- mobile-oriented layouts;
- event-centred information presentation;
- card-based browsing;

- quick access to the most relevant actions;
- consistent navigation within each role;
- explicit loading, error, empty-result, and status feedback;
- separation between normal-user, venue-manager, and PR-manager operations;
- reuse of the same backend data across different role-specific interfaces.

The normal-user interface focuses on event discovery, ticket acquisition, pregame coordination, saved events, friendships, notifications, and profile management.

The venue-manager interface focuses on venue administration, event creation and editing, promotion management, ticket inspection, and dashboard information.

The PR-manager interface focuses on assigned events, attributed tickets, commission-related information, and profile management.

The frontend communicates with the Spring Boot backend through the shared Axios-based `nightoutApi` client. Operational domain data is retrieved through REST APIs and is not stored as an independent frontend dataset.

The browser stores a lightweight session marker in `localStorage` to preserve frontend navigation after page refresh. This marker does not represent a server-side authenticated session or access token.

3.2 Navigation and Role-Based Routing

Frontend navigation is implemented through React Router.

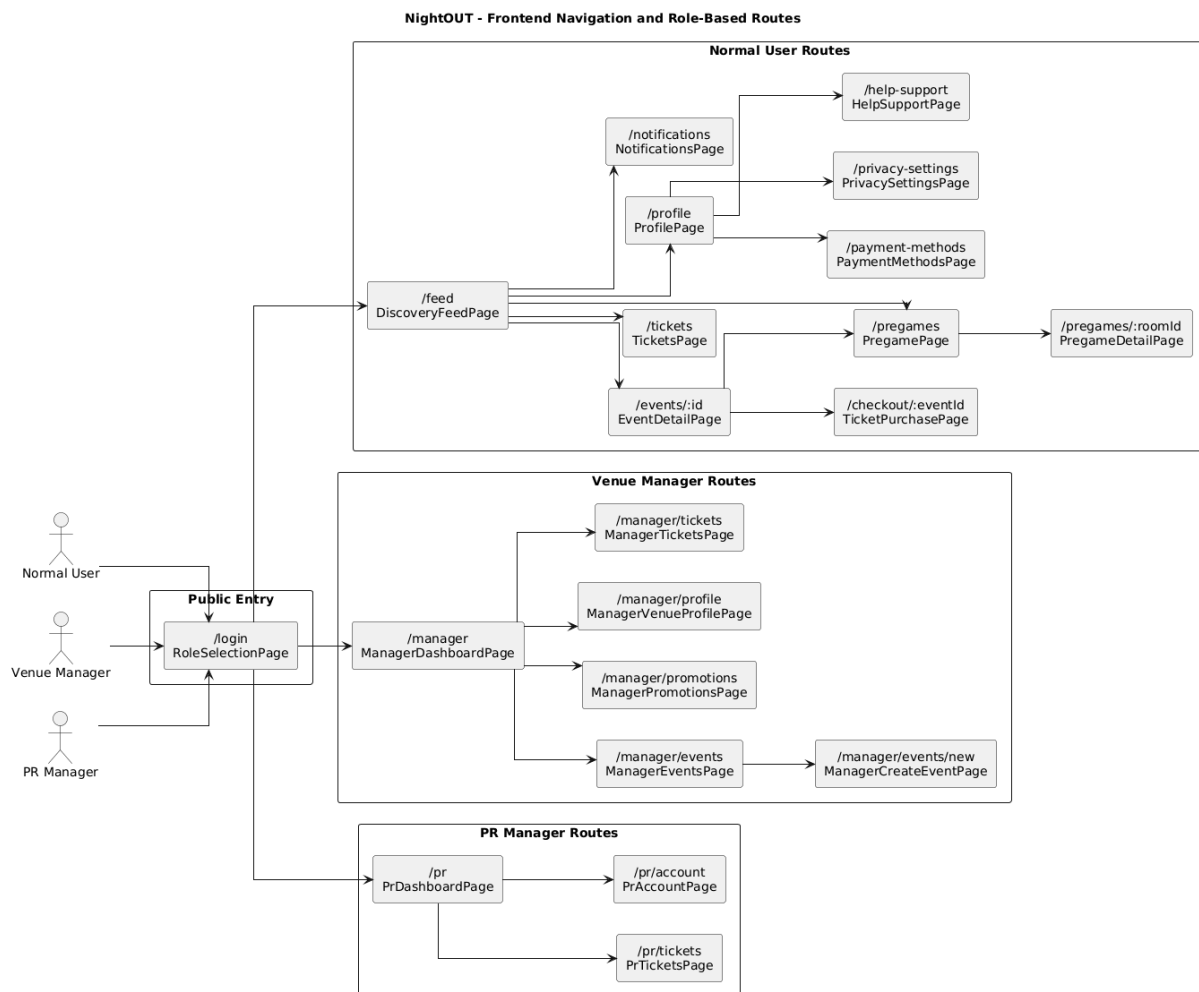
The public entry point is the login route. After successful credential verification, the application retrieves the user information and redirects the user according to the returned role:

- `NORMAL_USER` → `/feed`;
- `VENUE_MANAGER` → `/manager`;
- `PR_MANAGER` → `/pr`.

Route protection is implemented through frontend role guards. A page is rendered only when the session marker contains one of the roles accepted by the selected route.

If the current role is not allowed, the frontend redirects the user to the corresponding role home page or login page.

This mechanism provides role-specific navigation and prevents users from accidentally opening pages intended for another interface. However, it is not a backend authorization mechanism. The server does not currently derive the acting user from an authenticated principal.



@startuml

title NightOUT - Frontend Navigation and Role-Based Routes

left to right direction

skinparam componentStyle rectangle
 skinparam packageStyle rectangle
 skinparam shadowing false
 skinparam linetype ortho
 skinparam defaultTextAlignment center

actor "Normal User" as User
 actor "Venue Manager" as Manager
 actor "PR Manager" as PR

package "Public Entry" {
 component "/login\nRoleSelectionPage" as Login
 }

package "Normal User Routes" {
 component "/feed\nDiscoveryFeedPage" as Feed
 component "/events/:id\nEventDetailPage" as EventDetail
 component "/checkout/:eventId\nTicketPurchasePage" as Checkout

```
component "/tickets\nTicketsPage" as Tickets
component "/pregames\nPregamePage" as Pregames
component "/pregames/:roomId\nPregameDetailPage" as PregameDetail
component "/profile\nProfilePage" as Profile
component "/notifications\nNotificationsPage" as Notifications
component "/payment-methods\nPaymentMethodsPage" as Payment
component "/privacy-settings\nPrivacySettingsPage" as Privacy
component "/help-support\nHelpSupportPage" as Support
}
```

```
package "Venue Manager Routes" {
  component "/manager\nManagerDashboardPage" as ManagerDashboard
  component "/manager/events\nManagerEventsPage" as ManagerEvents
  component "/manager/events/new\nManagerCreateEventPage" as CreateEvent
  component "/manager/promotions\nManagerPromotionsPage" as Promotions
  component "/manager/profile\nManagerVenueProfilePage" as VenueProfile
  component "/manager/tickets\nManagerTicketsPage" as ManagerTickets
}
```

```
package "PR Manager Routes" {
  component "/pr\nPrDashboardPage" as PrDashboard
  component "/pr/tickets\nPrTicketsPage" as PrTickets
  component "/pr/account\nPrAccountPage" as PrAccount
}
```

User --> Login
Manager --> Login
PR --> Login

Login --> Feed : NORMAL_USER
Login --> ManagerDashboard : VENUE_MANAGER
Login --> PrDashboard : PR_MANAGER

Feed --> EventDetail
EventDetail --> Checkout
EventDetail --> Pregames
Feed --> Tickets
Feed --> Pregames
Feed --> Profile
Feed --> Notifications

Pregames --> PregameDetail

Profile --> Payment
Profile --> Privacy
Profile --> Support

ManagerDashboard --> ManagerEvents
ManagerDashboard --> Promotions
ManagerDashboard --> VenueProfile
ManagerDashboard --> ManagerTickets
ManagerEvents --> CreateEvent

PrDashboard --> PrTickets

PrDashboard --> PrAccount

note bottom of Login

The login page verifies credentials through the backend and stores a frontend session marker in localStorage.
end note

note bottom of Feed

Normal-user pages are protected by frontend role guards.
end note

note bottom of ManagerDashboard

Venue-manager pages are protected by frontend role guards.
end note

note bottom of PrDashboard

PR-manager pages are protected by frontend role guards.
end note

note bottom

Frontend role guards control navigation, but they do not provide server-side authentication or authorization.
end note

@enduml

3.3 Normal User interface

3.3.1 Login and Role Redirect

The login page is implemented by `RoleSelectionPage` and is available at `/login`.

Despite the component name, the current page performs credential-based login rather than simply selecting a mock role.

Main elements:

- email input;
- password input;
- login action;
- validation and authentication-error messages.

Main actions:

- submit credentials;
- receive the authenticated user's identifier, profile information, and role;
- store the frontend session marker;
- redirect to the home page associated with the returned role.

User registration is not currently available. All demo users are created by the backend startup dataset.

3.3.2 Event Discovery Feed and Filters

The event-discovery interface is implemented by [DiscoveryFeedPage](#) and is available at [/feed](#).

The page displays available nightlife events and personalized recommendations.

Main information:

- event image;
- event title;
- venue name and area;
- date and time;
- music genre;
- venue category;
- price or entry condition;
- stored popularity information;
- remaining availability;
- friends attending, when available;
- recommendation score and reasons.

Available search and filter controls include:

- text search;
- area;
- music genre;
- venue category;
- date or date range;
- minimum and maximum price;
- entry condition;
- featured-event selection;
- sorting by date, price, or popularity.

Main actions:

- browse events;
- modify search and filter criteria;
- open event details;
- inspect recommended events;
- navigate to tickets, pregames, notifications, or profile.

Filter changes produce new backend requests through `GET /api/events`. The result is displayed as a list of event cards.

The page provides:

- loading feedback while data is being retrieved;
- error feedback when an API call fails;
- an explicit empty-result message when no event satisfies the selected criteria.

The selectable areas, genres, categories, and entry conditions are currently constrained by frontend constants. Event data itself is retrieved from the backend.

3.3.3 Event Detail Page

The event-detail interface is implemented by `EventDetailPage` and is available at `/events/:id`.

The page retrieves the complete event representation through the backend.

Main information:

- event title and image;
- venue name, category, address, area, and location;
- date and time;
- music genre;
- dress code;
- target age;
- price and VIP price;
- entry conditions;
- description;
- event capacity and remaining availability;
- stored popularity and event-related scores;
- active promotions;
- friends attending;
- associated pregame rooms;
- available return-transport information.

Main actions:

- save or unsave the event;
- start the ticket-purchase flow;
- open the list of associated pregames;
- open map or sharing links;
- inspect promotions and social information.

Saving an event creates or updates a persisted `EventParticipation` record. The saved state therefore remains available after page refresh while the backend is running.

The page does not directly place the user in the waiting list. The final status is determined during the ticket-confirmation operation.

3.3.4 Ticket Purchase and Confirmation

The ticket-purchase interface is implemented by `TicketPurchasePage` and is available at `/checkout/:eventId`.

The interface follows the two-step backend ticket lifecycle.

First, the user requests a ticket. The backend creates a `PENDING` ticket.

The user then confirms the checkout operation. Depending on event capacity, the ticket becomes:

- `CONFIRMED`, when capacity is available;
- `WAITING_LIST`, when the event is full;
- `EXPIRED`, when the confirmation deadline is no longer valid.

Main information:

- selected event;
- ticket type;
- ticket price;
- applied discount or promotion information;
- payment-method interface;
- current ticket status;
- confirmation deadline;
- confirmation or waiting-list result.

Main actions:

- request a ticket;
- confirm checkout;
- cancel a pending ticket;
- inspect the current ticket status.

The page periodically refreshes pending-ticket information while the checkout is active.

Payment information is part of the prototype interface. No external payment provider is currently integrated.

3.3.5 My Tickets Page

The ticket-overview interface is implemented by `TicketsPage` and is available at `/tickets`.

The page displays all tickets associated with the current user.

Main information:

- event name;
- venue;
- event date and time;
- ticket type;
- price;
- ticket status;
- digital QR payload for confirmed tickets;
- waiting-list state;
- cancellation or expiration state.

Main actions:

- inspect ticket details;
- cancel an active ticket;
- display the digital confirmation associated with a confirmed ticket.

Ticket status is retrieved from the backend and is not maintained only as frontend state.

3.3.6 Pregame List, Creation and Detail

Pregame functionality is implemented through two main routes:

- `/pregames`, implemented by `PregamePage`;
- `/pregames/:roomId`, implemented by `PregameDetailPage`.

The pregame-list page allows users to:

- browse available rooms;
- filter rooms by event;
- search and sort rooms;
- create a new pregame room;
- join a room;
- delete a room when acting as its host;
- open the complete room details.

Main room information:

- title;
- linked event;
- host;
- meeting location;
- meeting time;
- maximum participants;
- current participant count;
- description;
- official-room indicator;
- room image.

The pregame-detail page displays:

- complete room information;
- current participant list;
- available capacity;
- location link;
- current-user membership state.

Main detail-page actions:

- join the room;
- leave the room;
- inspect the participants;
- open the meeting location.

Membership and room-capacity updates are persisted by the backend.

The frontend validates some form conditions, such as meeting time and participant values. Some of these validations are not yet repeated consistently by the backend.

3.3.7 User Profile and Friendships

The user-profile interface is implemented by `ProfilePage` and is available at `/profile`.

Main information:

- name and profile information;
- avatar;
- city and location-related information;
- music preferences;
- saved events;
- ticket and event-related statistics;
- links to privacy, payment, and support settings.

Main actions:

- edit profile data;
- update music preferences;
- upload or remove the avatar;
- inspect saved events;
- access account settings.

Friendship management is embedded in the profile through `FriendsSection`.

Friendship functionality includes:

- search for another user;
- send a friendship request;
- view received requests;

- accept or reject a request;
- view accepted friends;
- remove a friendship.

Accepted friendship data is also used in the event feed, event-detail page, and recommendation engine to show or evaluate friends attending an event.

The current implementation does not contain a dedicated social activity-feed page. It does not display a chronological feed of friends saving or attending events.

3.3.8 Notifications

The notification interface is implemented by [NotificationsPage](#) and is available at [/notifications](#).

Main information:

- notification type;
- notification message;
- creation time;
- read or unread state.

Main actions:

- retrieve user notifications;
- inspect notification content;
- mark a notification as read.

The read state is persisted through the backend.

Runtime-generated notifications currently include:

- ticket requests and ticket-status changes;
- waiting-list promotion;
- friendship requests;
- accepted friendships;
- pregame updates.

Event-update notifications and friend-joined-event notifications are not currently generated by the live application flow. Seeded examples or corresponding enum values may still be displayed in the prototype.

3.3.9 Privacy, Payment Methods and Support

Privacy Settings

[PrivacySettingsPage](#), available at [/privacy-settings](#), allows the user to manage profile visibility and friendship-related preferences.

The updated settings are persisted through the backend.

Payment Methods

`PaymentMethodsPage`, available at `/payment-methods`, allows the user to:

- retrieve saved demo payment methods;
- create a payment method;
- select the default method;
- delete a method.

Payment methods are stored in H2 but do not represent a real payment-provider integration.

Help and Support

`HelpSupportPage`, available at `/help-support`, allows the user to create a support request.

The submitted subject and message are persisted by the backend as a `SupportRequest`.

3.4 Venue Manager Interface

Venue managers access a dedicated group of routes under `/manager`.

The interface supports dashboard visualization, venue editing, event creation and management, promotion management, and ticket inspection.

Venue-manager routes are protected by frontend role guards. Backend services also perform some role and ownership checks, but the application does not currently use an authenticated server-side principal.

3.4.1 Manager Dashboard

The venue-manager dashboard is implemented by `ManagerDashboardPage` and is available at `/manager`.

Main information:

- managed venues;
- events associated with the selected manager;
- ticket and attendance-related indicators;
- pregame information;
- sales-channel statistics;
- demonstration insight cards.

Main actions:

- navigate to managed events;
- create an event;
- open venue-profile management;
- manage promotions;
- inspect event tickets.

Dashboard data is retrieved from `GET /api/manager/dashboard`.

In the current implementation, the frontend does not pass the logged-in manager identifier in this request. When the identifier is absent, the backend selects the first venue manager in the database.

Some displayed metrics combine persisted ticket counts with seeded `SalesChannel` totals, while some insights are fixed demonstration values. The dashboard must therefore be considered a prototype statistical interface rather than a fully consistent analytics system.

3.4.2 Event Management

Event management is implemented by `ManagerEventsPage` and is available at `/manager/events`.

Main information:

- events created by the selected manager;
- venue;
- event date and time;
- genre;
- capacity;
- price;
- current event information.

Main actions:

- search or locally filter managed events;
- open the event-creation page;
- edit an existing event;
- update event information;
- update event capacity;
- delete an event.

Event editing is embedded directly inside `ManagerEventsPage`. There is no separate event-edit route.

Updated events are persisted in the shared backend database and therefore become visible to the normal-user interface.

3.4.3 Event Creation

Event creation is implemented by `ManagerCreateEventPage` and is available at `/manager/events/new`.

Main input fields:

- title;
- selected venue;
- date and time;
- music genre;
- description;
- dress code;
- target age;
- entry condition;
- standard price;
- VIP price;
- capacity;
- image-related information;
- featured status.

Main actions:

- retrieve the venues available to the manager;
- validate the event form;
- create the event;
- display validation or API errors.

The created event is persisted in the same `EventRepository` used by normal-user discovery.

The current backend verifies the supplied manager role but does not consistently verify that the selected venue belongs to that manager.

After successful creation, the current frontend navigates to `/events/{id}`. Since that route is restricted to normal users, the venue manager is immediately redirected away from the created-event detail page.

3.4.4 Promotion Management

Promotion management is implemented by `ManagerPromotionsPage` and is available at `/manager/promotions`.

Main information:

- promotion label;
- type;
- description;

- promotion code;
- discount value;
- validity interval;
- active or inactive state;
- associated venue;
- optional associated event.

Main actions:

- retrieve promotions for a managed venue;
- create a promotion;
- edit a promotion;
- activate or deactivate a promotion;
- delete a promotion.

Promotion changes are persisted by the backend and can be included in normal-user event details.

3.4.5 Venue Profile

Venue-profile management is implemented by [ManagerVenueProfilePage](#) and is available at </manager/profile>.

Main information:

- managed venues;
- venue name;
- category;
- address and area;
- description;
- image and contact information;
- partner and rating information.

Main actions:

- select a managed venue;
- edit venue information;
- persist the updated venue profile.

The backend verifies that the selected venue is assigned to the supplied manager before applying the venue update.

3.4.5 Manager Tickets

Manager ticket inspection is implemented by [ManagerTicketsPage](#) and is available at </manager/tickets>.

The page displays tickets associated with events created by the selected manager.

Main information:

- ticket owner;
- event;
- ticket type;
- ticket status;
- price;
- creation and confirmation information;
- PR attribution, when present;
- check-in-related information.

The page is intended for inspection and demonstration of event attendance data. The acting manager is currently identified through a frontend-supplied identifier.

3.5 PR Manager Interface

PR managers access a dedicated group of routes under [/pr](#).

PR accounts are represented by [AppUser](#) instances with the [PR_MANAGER](#) role.

The PR interface focuses on assigned events, attributed ticket activity, commission information, and profile management.

3.5.1 PR Dashboard

The PR dashboard is implemented by [PrDashboardPage](#) and is available at [/pr](#).

Main information:

- events assigned to the PR manager;
- attributed ticket totals;
- revenue or commission-related indicators;
- event-level performance information.

Main actions:

- inspect assigned-event information;
- navigate to attributed tickets;
- open the PR account page.

Dashboard data is retrieved from the backend through the PR dashboard API.

3.5.2 PR Tickets

The PR ticket interface is implemented by [PrTicketsPage](#) and is available at [/pr/tickets](#).

The page displays tickets attributed to the selected PR manager.

Main information:

- ticket owner;
- associated event;
- status;
- ticket type;
- price;
- applied code or PR attribution;
- commission-related values.

The current prototype uses a limited number of seeded PR assignments and attributed tickets.

3.5.3 PR Account

The PR account interface is implemented by [PrAccountPage](#) and is available at [/pr/account](#).

Main information:

- PR profile data;
- avatar;
- account-related information;
- summary information associated with the PR role.

Main actions:

- retrieve the PR profile;
- update profile information;
- upload or remove the avatar;
- navigate to the PR dashboard and attributed tickets.

3.6 Current limitation

4. Requirements Traceability

4.1 Traceability Approach and Status Definitions

This section maps the requirements defined in the Requirements Analysis and Specification Document to the corresponding design and implementation elements of NightOUT.

The Requirements Analysis and Specification Document does not assign explicit alphanumeric identifiers to individual requirements. Therefore, this Design Document introduces stable requirement identifiers based on the categories and ordering used in Section 2.6 of the RASD.

The prefixes used in the traceability matrices are:

- **FR-US** : User and Profile Requirements;
- **FR-EV** : Event Discovery Requirements;
- **FR-REC** : Recommendation Requirements;
- **FR-RS** : Reservation and Ticketing Requirements;
- **FR-PR** : Pregame Requirements;
- **FR-SO** : Social Layer Requirements;
- **FR-NO** : Notification Requirements;
- **FR-VM** : Venue Manager Requirements;
- **NFR-USAB** : Usability Requirements;
- **NFR-PERF** : Performance Requirements;
- **NFR-REL** : Reliability Requirements;
- **NFR-ROB** : Robustness Requirements;
- **NFR-MAIN** : Maintainability Requirements;
- **NFR-SEC** : Security and Privacy Requirements.

Traceability is evaluated against the current implementation baseline. The matrix distinguishes between functionality that is completely available, functionality implemented through a design different from the initial proposal, partially implemented functionality, mock or seed-only behavior, functionality not implemented, and requirements that have not yet been verified through appropriate tests.

4.2 Functional Requirements Traceability

4.2.1 User and Profile Requirements

ID	RASD requirement	Current design and implementation elements	Status	Traceability notes
FR-US-01	User Login	RoleSelectionPage, AuthController, AuthenticationService, AppUserRepository, frontend session context	Implemented differently	Credentials are verified by the backend and the user is redirected according to UserRole. The frontend stores a localStorage marker, but there is no server-side session or token.

FR-US-02	User Profile Visualization	<code>ProfilePage,</code> <code>UserController,</code> <code>UserService,</code> <code>UserDataMediator,</code> <code>AppUser,</code> profile DTOs	Implemented	The user can retrieve and display profile information, preferences, avatar, saved events and profile-related statistics.
FR-US-03	User Preferences	<code>ProfilePage,</code> <code>UpdateProfileRequest,</code> <code>UserService,</code> <code>AppUser.musicPreferences</code>	Implemented	Music preferences are editable and persisted. They are stored as strings rather than as a typed <code>MusicGenre</code> collection.
FR-US-04	Saved Events	<code>EventDetailPage,</code> saved-event endpoints, <code>UserService,</code> <code>EventParticipation,</code> <code>EventParticipationRepository</code>	Implemented	Saving an event creates or updates a persisted participation record with the <code>SAVED</code> value.
FR-US-05	View Saved Events	<code>ProfilePage,</code> <code>DiscoveryFeedPage,</code> <code>GET /api/users/{userId}/saved-events,</code> <code>UserService</code>	Implemented	Saved events are retrieved from the backend and displayed in the profile and feed-related interfaces.

4.2.2 Event Discovery Requirements

ID	RASD requirement	Current design and implementation elements	Status	Traceability notes
FR-EV-01	Browse Events	<code>DiscoveryFeedPage,</code> <code>EventController,</code> <code>EventService,</code> <code>EventDataMediator,</code> <code>EventRepository</code>	Implemented	The frontend retrieves and displays available events through <code>GET /api/events</code> .

FR-EV-02	View Event Details	<code>EventDetailPage</code> , <code>GET /api/events/{eventId}</code> , <code>EventService</code> , <code>NightOutMapper</code>	Implemented	The page displays aggregate event, venue, ticket, promotion, pregame and transport information.
FR-EV-03	Event Information Completeness	<code>Event</code> , <code>Venue</code> , <code>EventDetailDto</code> , <code>EventDetailPage</code>	Implemented	Title, venue, location, date/time, genre, dress code, price or entry conditions and capacity information are available.
FR-EV-04	Filter Events	<code>DiscoveryFeedPage</code> , <code>EventController</code> , <code>EventService.findEvents</code>	Implemented differently	All required filtering criteria are available, but filtering uses an in-memory conditional pipeline rather than separate <code>FilterStrategy</code> implementations.
FR-EV-05	Empty Filter Result	<code>DiscoveryFeedPage</code> empty-result rendering	Implemented	The frontend displays an explicit no-results state when the backend returns an empty list.
FR-EV-06	Event Popularity View	<code>Event.popularityScore</code> , popularity sorting in <code>EventService</code> , event cards	Partially implemented	Events can be displayed or sorted using one stored popularity score. The system does not calculate or allow selection among the multiple popularity criteria described in the RASD, such as most saved, most booked or genre trends.

4.2.3 Recommendation Requirements

ID	RASD requirement	Current design and implementation elements	Status	Traceability notes
FR-REC-01	Recommended Event Feed	<code>DiscoveryFeedPage</code> , <code>RecommendationController</code> , <code>RecommendationService</code> , <code>RecommendationEngine</code>	Implemented	The feed retrieves ranked recommendations with score and explanation reasons.

FR-REC-02	Music-Based Recommendation	<code>MusicPreferenceStrategy</code> , <code>AppUser.musicPreferences</code>	Implemented	Music preferences contribute to candidate-event scores.
FR-REC-03	Social-Based Recommendation	<code>FriendsAttendingStrategy</code> , <code>FriendshipRepository</code> , confirmed tickets	Implemented	Accepted friendships and confirmed event attendance contribute to recommendations. The untouched startup dataset has limited live friendship data.
FR-REC-04	Distance-Based Recommendation	<code>DistanceStrategy</code> , <code>GeographicDistanceService</code> , user and venue coordinates	Implemented	Geographic distance is calculated from stored coordinates and contributes to ranking.

4.2.4 Reservation and Ticketing Requirements

ID	RASD requirement	Current design and implementation elements	Status	Traceability notes
FR-RS-01	Request Reservation	<code>TicketPurchasePage</code> , <code>POST /api/tickets</code> , <code>TicketController</code> , <code>TicketService</code> , <code>Ticket</code>	Implemented differently	The implementation uses <code>Ticket</code> instead of <code>Reservation</code> . A request first creates a <code>PENDING</code> ticket, followed by a separate confirmation operation.

FR-RS-02	Confirm Reservation with Available Capacity	POST <code>/api/tickets/{id}/confirm</code> , <code>TicketService</code> , <code>TicketStateFactory</code> , <code>PendingTicketState</code>	Implemented	Confirmed-ticket count is compared with event capacity; available capacity produces CONFIRMED .
FR-RS-03	Waiting List for Full Events	<code>TicketService.confirmTicket</code> , <code>PendingTicketState.moveToWaitingList</code> , <code>TicketStatus.WAITING_LIST</code>	Implemented	A full event causes the pending ticket to enter the waiting list.
FR-RS-04	View Reservation Status	<code>TicketsPage</code> , <code>TicketPurchasePage</code> , ticket retrieval endpoints, <code>TicketDto</code>	Implemented	The frontend displays pending, confirmed, waiting-list, cancelled and expired statuses.
FR-RS-05	Cancel Reservation	DELETE <code>/api/tickets/{ticketId}</code> , <code>TicketService.cancelTicket</code> , concrete ticket states	Implemented	Active tickets can be cancelled and the new status is persisted. Resource ownership is not bound to an authenticated principal.
FR-RS-06	Promote Waiting List Reservation	<code>TicketService.promoteFirstWaitingTicket</code> , ordered repository query, <code>WaitingListTicketState</code>	Implemented	The first waiting ticket ordered by createdAt is promoted after a confirmed ticket cancellation.

FR-RS-07	Prevent Duplicate Active Reservations	<code>TicketService</code> active-ticket validation, <code>TicketRepository</code>	Implemented	Duplicate active requests are rejected. The rule is currently enforced at service level rather than through a database-level concurrent constraint.
FR-RS-08	Reservation Status Values	<code>TicketStatus</code> , <code>TicketState</code> , five concrete State classes	Implemented	All five RASD states are represented.
FR-RS-09	Digital Reservation Confirmation	<code>Ticket.qrPayload</code> , <code>TicketDto</code> , <code>TicketsPage</code> , <code>TicketPurchasePage</code>	Implemented	Confirmed tickets display a generated digital QR payload. It is a simulated confirmation and not a legally valid external ticket.

4.2.5 Pregame Requirements

ID	RASD requirement	Current design and implementation elements	Status	Traceability notes
FR-PR-01	Create Pregame Room	<code>PregamePage</code> , <code>POST /api/pregames</code> , <code>PregameController</code> , <code>PregameService</code>	Implemented	A room is created for an existing event and the host is automatically included as a participant.
FR-PR-02	Pregame Room Information	<code>PregameRoom</code> , <code>PregameRoomDto</code> , <code>PregameDetailPage</code>	Implemented	Host, event, location, time, maximum participants, participants and description are stored.

FR-PR-03	Browse Pregame Rooms	<code>/pregames, /events/{eventId}/pregames, PregamePage, PregameDetailPage</code>	Implemented	Rooms can be browsed globally or in relation to an event.
FR-PR-04	Join Pregame Room	join endpoint, <code>PregameService.joinPregame</code>	Implemented	The backend verifies capacity and membership before persisting the participant.
FR-PR-05	Prevent Joining Full Room	capacity validation in <code>PregameService</code>	Implemented	Full-room join attempts return an HTTP error.
FR-PR-06	Leave Pregame Room	leave endpoint, <code>PregameService.leavePregame</code>	Implemented	A participant can leave and the room is persisted. Host-leave semantics remain undefined.
FR-PR-07	Prevent Duplicate Participation	membership validation in <code>PregameService</code>	Implemented	The same user cannot be added twice to the participant collection.

4.2.6 Social Layer Requirements

ID	RASD requirement	Current design and implementation elements	Status	Traceability notes
FR-SO-01	Friends Relation Creation	<code>FriendsSection, FriendshipController, FriendshipService, Friendship, FriendshipStatus</code>	Implemented differently	The current workflow uses friendship requests, acceptance and rejection. It replaces the older generic <code>SocialRelation</code> model.
FR-SO-02	View Friends	<code>FriendsSection, friends endpoint, FriendshipService</code>	Implemented	Accepted friends and received requests are displayed in the user profile.

FR-SO-03	Event Participation Status	<code>EventParticipation</code> , save/unsave endpoints, confirmed tickets	Partially implemented	Users can save events and confirmed attendance is represented by tickets. There is no complete API for selecting generic interested/attending participation statuses.
FR-SO-04	Friends Attending Event	friends-attending endpoint, <code>EventDetailPage</code> , <code>DiscoveryFeedPage</code> , confirmed tickets	Partially implemented	Friends with confirmed tickets can be displayed. The system does not provide the complete “attending or interested” behavior described in the RASD.
FR-SO-05	Activity Feed	No dedicated route, service, controller or page	Not implemented	Friendship management and event social indicators exist, but there is no chronological social activity feed.

4.2.7 Notification Requirements

ID	RASD requirement	Current design and implementation elements	Status	Traceability notes
FR-NO-01	Reservation Notification	ticket application events, <code>TicketNotificationObserver</code> , <code>NotificationService</code>	Implemented	Ticket requests and status changes create persisted notifications.
FR-NO-02	Waiting List Notification	waiting-list promotion event, ticket observer, <code>UserNotification</code>	Implemented	The promoted user receives a persisted waiting-list availability notification.
FR-NO-03	Friend Event Notification	<code>FRIEND_JOINED_EVENT</code> notification type and seeded notification	Partially implemented	A seeded example can be displayed, but live friend join/save operations do not publish this notification.
FR-NO-04	Event Update Notification	<code>NotificationType.EVENT_UPDATE</code>	Not implemented	The notification type exists, but event creation/update/cancellation services do not generate the required notification.

FR-NO-05	View Notification s	<code>NotificationsPage</code> , notification retrieval endpoint, <code>NotificationService</code>	Implemented	User notifications are retrieved and displayed.
FR-NO-06	Mark Notification as Read	read endpoint, <code>NotificationsPage</code> , <code>UserNotification.read</code>	Implemented	Read status is updated and persisted.

4.2.8 Venue Manager Requirements

ID	RASD requiremen t	Current design and implementation elements	Status	Traceability notes
FR-VM-01	Venue Manager Login	<code>RoleSelectionPage</code> , <code>AuthController</code> , <code>AuthenticationService</code> , <code>UserRole.VENUE_MANAGER</code>	Implemented differently	Credential verification and role redirect exist, but there is no server-side authenticated venue-manager session.
FR-VM-02	Create Event	<code>ManagerCreateEventPage</code> , <code>POST</code> <code>/api/manager/events</code> , <code>VenueManagementService</code>	Partially implemented	Event creation works and events become visible to users. The backend does not verify that the selected venue belongs to the supplied manager, although the RASD requires creation only for managed venues.
FR-VM-03	Update Event	<code>ManagerEventsPage</code> , update endpoint, <code>VenueManagementService</code>	Partially implemented	Event creator checks exist, but the manager can select a venue assigned to another manager during an update.
FR-VM-04	Manage Event Capacity	event create/update DTOs and manager forms	Implemented	Capacity can be defined and updated. No explicit rule prevents reducing capacity below the current confirmed-ticket count.

FR-VM-05	Create Promotion	<code>ManagerPromotionsPage,</code> <code>PromotionController,</code> <code>PromotionService</code>	Implemented	The implementation provides complete creation, update, activation and deletion operations.
FR-VM-06	View Event Statistics	<code>ManagerDashboardPage</code> <code>, DashboardService,</code> <code>DashboardDataMediator</code>	Partially implemented	A dashboard exists, but it does not reliably select the logged-in manager and does not provide a consistent set of saved, reservation and waiting-list statistics. Some metrics mix separate seed populations or use hardcoded insights.

4.3 Traceability Discussion

The traceability analysis shows that the core NightOUT prototype is substantially implemented, particularly in the areas of event discovery, event details, ticket lifecycle management, waiting-list promotion, pregame membership, personalized recommendations, saved events, profile management, and venue-manager event and promotion operations.

The ticketing requirements are implemented through a design evolution. The implementation uses `Ticket` rather than `Reservation` and separates ticket request from checkout confirmation. The required lifecycle statuses remain represented through `TicketStatus` and concrete runtime State implementations.

Recommendation requirements are mapped to a genuine Strategy-based implementation. Music preferences, social context, and geographic distance are represented by separate strategy components. The implementation also provides booking-history, saved-event, and popularity contributions as additional recommendation features beyond the individually enumerated RASD requirements.

Event filtering is functionally available but does not use the Strategy pattern proposed by the initial design. Filtering criteria are applied through conditional operations in `EventService`. Similarly, event popularity is currently represented by a stored score rather than calculated through multiple popularity strategies.

The pregame requirements are broadly implemented. Creation, browsing, joining, leaving, duplicate prevention, and capacity checks are persisted by the backend. Some additional domain constraints, such as the behavior of a room host leaving the room, remain undefined.

The social layer is only partially complete. Friendship requests and accepted friendships are operational, and confirmed friends attending an event can be displayed. However, generic

interested/attending participation states and the social activity feed described in the RASD are not currently available.

Notification viewing, read-state updates, ticket notifications, waiting-list promotion notifications, friendship notifications, and pregame notifications are implemented. Friend event participation notifications are represented only by seeded examples, while event-update notifications do not have a runtime producer.

Venue-manager functionality is operational but does not completely satisfy the ownership and statistics requirements. Event creation and update are available, but the selected venue is not consistently verified as belonging to the acting manager. The manager dashboard also omits the logged-in manager identifier in the current frontend request and combines statistical values from different synthetic data sources.

The main nonfunctional gap is backend security. Role-based route guards exist in the frontend, but there is no server-side authenticated principal, token, session, or Spring Security authorization layer. Consequently, user and manager identifiers supplied by the client are trusted by the backend.

Performance requirements have not yet been verified through repeatable measurements. Reliability is supported under normal sequential prototype use, but concurrent ticket requests and overbooking prevention have not been validated through locking mechanisms or concurrency tests.

The requirements matrix should therefore be interpreted as an as-built traceability assessment rather than as a claim that all RASD requirements have been fully verified.

5. Implementation, Integration and Test Plan

This section describes the dependency-based implementation order of NightOUT, the current integration status of its components, the verification activities already performed, the remaining testing plan, and the synthetic data used by the prototype.

Although the main application is already implemented and runnable, some verification and stabilization activities remain incomplete. Therefore, this section distinguishes between:

- implemented and integrated functionality;
- automated tests currently available;
- manual runtime verification performed on the current implementation;
- testing activities still required for complete regression coverage;
- test-data scenarios currently available or still missing.

The implementation and integration order presented below is a logical dependency order derived from the current architecture. It describes how the system components depend on

one another and does not claim to reproduce the exact chronological order of every repository commit.

5.1 Implementation Order

NightOUT follows an incremental implementation approach. Foundational infrastructure and persistent domain entities are required before application services and frontend use cases can be completed.

The current implementation can be organized into the following dependency-based phases.

Step 1: Project Infrastructure and Persistence

he first phase establishes the technical foundation of the application.

Main elements:

- Java Spring Boot backend;
- React and TypeScript frontend;
- Maven backend configuration;
- npm and Vite frontend configuration;
- Spring Data JPA;
- H2 in-memory database;
- CORS configuration;
- global API exception handling;
- DTO classes and mapping infrastructure;
- startup data loading through `DemoDataLoader` and the workbook-derived JSON dataset.

This phase provides the execution environment and shared persistence infrastructure required by all subsequent components.

Step 2: User Accounts, Roles and Profiles

The second phase introduces the common account model and role-specific navigation.

Main elements:

- `AppUser`;
- `UserRole`;
- `AuthenticationService`;
- `AuthController`;
- `UserService`;
- `UserController`;
- `UserDataMediator`;
- `AppUserRepository`;

- frontend session context;
- login and role-based routing;
- profile, avatar, preferences and privacy management.

Normal users, venue managers, and PR managers are represented by the same persistent user entity and are distinguished by role.

Authentication currently consists of backend credential verification and a frontend `localStorage` session marker. Server-side sessions or token-based authentication are not part of the current prototype.

Step 3: Venue, Event and Event-Discovery Management

Events and venues form the central application dataset and are required by ticketing, pregames, promotions, recommendations and manager functionality.

Main elements:

- `Venue`;
- `Event`;
- `MusicGenre`;
- `VenueCategory`;
- `EventController`;
- `EventService`;
- `VenueManagementService`;
- `EventDataMediator`;
- `VenueManagementDataMediator`;
- `EventRepository`;
- `VenueRepository`;
- `NightOutMapper`;
- event feed;
- event-detail page;
- manager event creation and editing.

Event searching and filtering are included in this phase. They are implemented through conditional operations in `EventService`, rather than through the filtering Strategy hierarchy proposed in the initial design.

Step 4: Ticket Lifecycle and Capacity Management

Ticketing is implemented after events because every ticket is associated with a user and an event.

Main elements:

- `Ticket`;
- `TicketStatus`;

- `TicketController;`
- `TicketService;`
- `TicketDataMediator;`
- `TicketRepository;`
- `TicketState;`
- `TicketStateFactory;`
- `PendingTicketState;`
- `ConfirmedTicketState;`
- `WaitingListTicketState;`
- `CancelledTicketState;`
- `ExpiredTicketState;`
- `TicketExpirationScheduler;`
- ticket-purchase and ticket-overview pages.

This phase implements pending-ticket creation, confirmation, event-capacity checks, waiting-list admission, cancellation, expiration, FIFO waiting-list promotion and digital QR confirmation.

The implementation uses `Ticket` instead of the `Reservation` class proposed by the initial design

Step 5: Notification Management

The notification component is integrated after the first domain operations capable of producing user-visible updates.

Main elements:

- `UserNotification;`
- `NotificationType;`
- `NotificationController;`
- `NotificationService;`
- `NotificationDataMediator;`
- `UserNotificationRepository;`
- ticket notification events and listeners;
- friendship notification events and listeners;
- notification page and read-state management.

Ticket and friendship notifications use Spring application events and listeners. Pregame notifications are integrated through direct calls to `NotificationService`.

Event-update and live friend-event-participation notification producers remain incomplete.

Step 6: Pregame Room Management

Pregame functionality depends on existing users and events.

Main elements:

- `PregameRoom`;
- `PregameController`;
- `PregameService`;
- `PregameDataMediator`;
- `PregameRoomRepository`;
- pregame list and creation page;
- pregame detail page.

This phase implements room creation, event association, participant management, duplicate-participation prevention, capacity checking, joining, leaving and host-controlled deletion.

Step 7: Saved Events and Friendship Features

The social component is introduced after users and events are available.

Main elements:

- `EventParticipation`;
- `Friendship`;
- `FriendshipStatus`;
- `FriendshipController`;
- `FriendshipService`;
- `FriendshipDataMediator`;
- `FriendshipRepository`;
- `EventParticipationRepository`;
- `FriendsSection`;
- saved-event operations;
- friends-attending event information.

The live friendship workflow replaces the generic `SocialRelation` model proposed in the initial design.

Saved events and confirmed friends attending an event are available. General interested/attending participation management and the social activity feed are not currently implemented.

Step 8: Personalized Recommendations and Stored Popularity

The recommendation component depends on user preferences, event information, saved events, ticket history, friendship data and geographic coordinates.

Main elements:

- `RecommendationController`;

- `RecommendationService;`
- `RecommendationDataMediator;`
- `RecommendationEngine;`
- `RecommendationStrategy;`
- `MusicPreferenceStrategy;`
- `FriendsAttendingStrategy;`
- `DistanceStrategy;`
- `BookingHistoryStrategy;`
- `SavedEventsStrategy;`
- `PopularityStrategy.`

All applicable strategies contribute to a combined recommendation score.

Event popularity itself is not calculated by a dedicated strategy. It is currently stored as an attribute of the event and used for display, sorting and recommendation contribution.

Step 9: Venue-Manager, Promotion and PR Extensions

Role-specific operational functionality is added after the main event and ticketing infrastructure.

Main elements:

- manager dashboard;
- manager event list and editing;
- manager event creation;
- venue-profile editing;
- manager ticket inspection;
- promotion creation, update, activation and deletion;
- PR dashboard;
- PR event assignments;
- PR-attributed tickets and commission data;
- PR account management.

The current implementation also adds privacy settings, demo payment methods, support requests, return-transport placeholders and avatar file storage.

Step 10: Stabilization, Verification and Documentation

The final phase covers build verification, regression testing, data validation, defect correction and documentation alignment.

Current activities include:

- backend compilation and automated-test execution;
- frontend production build;
- runtime API verification;

- requirements-to-implementation audit;
- Design Document update.

Remaining high-priority activities include:

- extending automated test coverage;
- correcting manager venue-ownership validation;
- correcting manager dashboard identity and metric sources;
- correcting manager post-creation navigation;
- resolving frontend lint errors;
- restoring reliable Maven Wrapper execution;
- adding backend authentication and authorization before any external deployment.

5.2 Integration Plan

The integration plan follows the same incremental order.

Integration 1: Domain Entities, Repositories and H2

JPA domain entities are integrated with Spring Data repositories and the H2 in-memory database.

This integration is operational for users, venues, events, tickets, pregames, friendships, event participation, notifications, promotions, PR assignments, sales channels, payment methods, support requests and transport placeholders.

Data persists while the backend is running and is recreated when the application restarts because Hibernate uses `create-drop`.

Integration 2: Services, Focused Mediators and Repositories

Application services normally access persistence through focused data mediators.

Examples include:

- `TicketService` → `TicketDataMediator`;
- `EventService` → `EventDataMediator`;
- `PregameService` → `PregameDataMediator`;
- `FriendshipService` → `FriendshipDataMediator`;
- `RecommendationService` → `RecommendationDataMediator`;
- `DashboardService` → `DashboardDataMediator`.

`AuthenticationService` and `NightOutMapper` remain limited direct-repository exceptions.

Integration 3: REST Controllers and Application Services

REST controllers are connected to application services through validated request DTOs and response DTOs.

The backend currently exposes operations for:

- authentication and user profiles;
- events and recommendations;
- tickets;
- pregames;
- notifications;
- friendships;
- venue-manager operations;
- promotions;
- PR functionality;
- payment methods;
- support requests.

Invalid requests are translated into controlled HTTP error responses by the backend exception-handling mechanism.

Integration 4: React Frontend and REST Backend

The React frontend is connected to the Spring Boot backend through the shared Axios client.

All principal frontend pages retrieve operational domain data through REST requests rather than through an independent frontend event dataset.

The frontend production build succeeds. The API base URL is currently hardcoded for local execution.

Integration 5: Ticket Lifecycle, Capacity and Notifications

Ticket request, confirmation, cancellation and expiration are integrated with:

- persisted `TicketStatus`;
- runtime State objects;
- event-capacity counts;
- FIFO waiting-list queries;
- Spring ticket events;
- ticket notification listeners;
- persisted user notifications.

Cancellation of a confirmed ticket may promote the oldest waiting-list ticket and generate the corresponding notification.

Integration 6: Pregame, Event and Notification Components

Pregame rooms are linked to persistent events and are included in event-related DTOs.

Pregame list and detail pages retrieve the same backend room data. Join and leave operations update the room participant collection and create notifications.

Pregame notifications currently invoke `NotificationService` directly rather than using Spring application events.

Integration 7: Friendship, Event Attendance and Recommendations

Accepted friendships are integrated with confirmed tickets to determine friends attending an event.

Friendship data, saved-event records and ticket history are also used by the recommendation engine.

The startup dataset currently seeds legacy `SocialRelation` data rather than the `Friendship` entities used by the live feature. Consequently, social recommendation contributions may be absent until friendship records are created through the application.

Integration 8: Venue-Manager Administration and User Discovery

Manager-created events and normal-user event discovery use the same `EventRepository`.

New events therefore become visible through the global event feed and event-detail APIs.

Three integration defects remain:

- event creation and update do not consistently verify that the selected venue belongs to the acting manager;
- the manager dashboard frontend omits the logged-in manager identifier;
- successful manager event creation navigates to a normal-user-only route.

5.3 Testing Strategy

The testing strategy combines automated backend tests, build verification, manual runtime checks, and planned regression testing.

The current test suite is limited. Seven backend tests are available and pass, but the principal ticket, pregame, friendship, recommendation, manager and frontend scenarios are not covered by automated regression tests.

Manual API verification performed during the implementation audit provides evidence that several core flows work. However, manual checks do not replace repeatable automated tests.

No coverage percentage is reported because no coverage tool is currently configured.

5.3.1 Tests Currently Implemented

Test	Type	Verified behavior	Current result
<code>BackendApplicationTests#contextLoads</code>	Spring context integration smoke test	Application context and startup seed load successfully	Passed
<code>validUserCredentialsReturnUserRole</code>	MockMvc controller integration test	Normal-user credentials return the correct role	Passed
<code>validVenueCredentialsReturnVenueRole</code>	MockMvc controller integration test	Venue-manager credentials return the correct role	Passed
<code>validPrCredentialsReturnPrRole</code>	MockMvc controller integration test	PR-manager credentials return the correct role	Passed
<code>invalidPasswordReturnsUnauthorized</code>	MockMvc controller integration test	Wrong password produces HTTP 401	Passed
<code>unknownEmailReturnsUnauthorized</code>	MockMvc controller integration test	Unknown email produces HTTP 401	Passed
<code>loginResponseNeverIncludesPassword</code>	MockMvc controller integration test	The login response does not expose the password	Passed

5.3.2 Build and Static Verification

Verification	Result	Interpretation
Backend compilation and tests through downloaded Maven binary	Passed	Backend compiles; 7 tests passed with no failures or errors
Windows <code>mvnw.cmd test</code>	Failed before Maven startup	The Maven Wrapper launcher is not reproducible on the audited machine

Frontend production build	Passed	TypeScript compilation and Vite production bundling succeeded
Frontend lint	Failed with 27 errors and 1 warning	The configured frontend quality gate does not currently pass
Frontend automated tests	Not available	No test command or frontend test files are configured
Coverage measurement	Not available	No statement or branch coverage value can be reported

5.3.4 Planned Backend Unit and Service Testing

Ticket lifecycle tests

Tests should cover:

- request creates a **PENDING** ticket;
- available capacity produces **CONFIRMED**;
- full capacity produces **WAITING_LIST**;
- duplicate active ticket is rejected;
- expired pending ticket cannot be confirmed;
- confirmed ticket can be cancelled;
- waiting-list ticket can be cancelled;
- cancelled and expired tickets reject invalid transitions;
- FIFO waiting-list promotion;
- promotion notification creation;
- digital QR payload generation;
- confirmation-deadline behavior.

Pregame service tests

Tests should cover:

- valid room creation;
- host automatically added as participant;
- valid room join;
- duplicate join rejection;
- full-room rejection;
- valid leave;
- leaving a room without membership;
- host deletion;
- host-leave behavior;
- meeting time in the past;
- meeting time after the linked event;

- maximum-participant boundaries.

Event and manager tests

Tests should cover:

- valid event creation;
- missing venue or invalid data;
- manager can create an event only for an owned venue;
- manager cannot update another manager's event;
- manager cannot move an event to another manager's venue;
- event capacity update;
- event becomes visible through the global user endpoint;
- dashboard uses the selected manager;
- dashboard metrics use consistent source data;
- promotion ownership and event association.

Recommendation tests

Each strategy should be tested independently:

- matching and non-matching music preferences;
- accepted friend attending the candidate event;
- distance contribution;
- booking-history contribution;
- saved-event similarity contribution;
- stored-popularity contribution.

RecommendationEngine should also be tested to verify:

- additive score calculation;
- reason aggregation;
- candidate exclusion;
- result ordering;
- result limiting.

Friendship and notification tests

Tests should cover:

- friendship request creation;
- self-request rejection;
- duplicate request rejection;
- request acceptance and rejection;
- friend removal;
- friends-attending retrieval;
- notification creation through Spring events;
- notification retrieval;

- mark-as-read persistence;
- notification ownership after server-side authentication is introduced.

5.3.5 Black-Box and Integration Testing

Requirement area	Test case	Expected result
Authentication	Valid normal-user credentials	HTTP 200 and NORMAL_USER
Authentication	Invalid password	HTTP 401
Event filtering	Filter by genre, area, date and price	Every returned event satisfies all selected criteria
Event filtering	Criteria with no matching event	Empty list and frontend no-results state
Ticketing	Confirm ticket with available capacity	Ticket status is CONFIRMED
Ticketing	Confirm ticket for full event	Ticket status is WAITING_LIST
Ticketing	Request duplicate active ticket	HTTP 400
Waiting list	Cancel confirmed ticket with waiting list	First waiting ticket becomes CONFIRMED
Pregame	Join room with available capacity	Participant is added
Pregame	Join full room	HTTP 400
Pregame	Join same room twice	HTTP 400
Manager	Create valid event for owned venue	Event is created and visible to users
Manager authorization	Create event for another manager's venue	Request must be rejected
Friendship	Send and accept request	Users appear in each other's friend list
Notification	Mark notification as read	Read state remains persisted
Recommendation	Retrieve recommendations	Results are ordered and include reasons

5.3.6 White-Box and Boundary Value Testing

Feature	Relevant branches or boundaries
Ticket capacity	capacity 0, capacity 1, confirmed count $\text{capacity} - 1$, confirmed count capacity
Duplicate tickets	no ticket, cancelled ticket, pending ticket, confirmed ticket, waiting-list ticket
State transitions	every valid transition and every invalid transition
Waiting list	empty list, one waiting ticket, multiple waiting tickets with different timestamps
Pregame capacity	maximum 1, participant count $\text{max} - 1$, participant count max
Pregame membership	user absent, user present once, duplicate attempt
Event filtering	no result, one result, all events returned
Price	free event, minimum price, maximum configured price
Event creation	empty title, missing venue, past date, capacity 0, valid minimum capacity
Recommendation	no contribution, one strategy contribution, multiple contributions, tied scores
Notifications	unread, read, unknown identifier
Manager ownership	own venue, another manager's venue, unknown venue

Additional tests are required for simultaneous ticket operations.

The current service-level count and duplicate checks work during normal sequential execution, but no automated concurrency test proves that simultaneous requests cannot:

- exceed event capacity;
- create duplicate active tickets;
- promote more than one waiting ticket for the same available spot.

These tests should be introduced together with a transactional locking or persistence-level consistency policy.

5.3.7 Frontend Component and End-to-End Testing

The frontend currently has no automated test infrastructure.

A frontend test setup should be introduced using a React-compatible component-testing framework and an API-mocking layer.

Initial component tests should cover:

- login success and failure feedback;
- role-based redirection;
- event loading, error and empty states;
- filter form interaction;
- ticket-status rendering;
- waiting-list rendering;
- pregame join and leave actions;
- friendship request states;
- notification read-state updates;
- manager event form validation.

Planned E2E scenarios

E2E-01 — User Discovers and Confirms a Ticket

1. The user logs in.
2. The user opens the event feed.
3. The user applies event filters.
4. The user opens an event.
5. The user requests a ticket.
6. The user confirms the checkout.
7. The system displays a confirmed ticket and digital QR payload.

E2E-02 — Full Event and Waiting-List Promotion

1. The user selects a full event.
2. The user requests and confirms a ticket.
3. The ticket enters the waiting list.
4. Another confirmed ticket is cancelled.
5. The first waiting ticket is promoted.
6. The promoted user receives a notification.

E2E-03 — Pregame Membership

1. The user opens the pregame list.
2. The user selects a room with capacity.
3. The user joins the room.
4. The participant list is updated.
5. The user leaves the room.
6. The participant list is updated again.

E2E-04 — Venue Manager Creates an Event

1. The venue manager logs in.
2. The manager selects one of their venues.
3. The manager creates an event.
4. The event appears in the manager event list.
5. A normal user can retrieve the event in the feed.

E2E-05 — Friendship and Social Recommendation

1. A user sends a friendship request.
2. The second user accepts the request.
3. The friend obtains a confirmed ticket.
4. The event displays the friend as attending.
5. The social recommendation strategy contributes to the recommended-event score.

E2E-06 — Promotion and PR Attribution

1. A manager creates a promotion.
2. The promotion becomes visible in event details.
3. A ticket is created with PR attribution or promotion data.
4. The ticket appears in the corresponding PR ticket view.

5.4 Test Data

NightOUT uses a realistic synthetic startup dataset.

Data is loaded from two main sources:

- `nightout-workbook-seed.json`, derived from the project workbook;
- programmatic data created by `DemoDataLoader`.

The frontend does not maintain a separate operational domain dataset. It retrieves the startup data through backend APIs.

The database is an in-memory H2 database configured through Hibernate `create-drop`. Therefore:

- the dataset is recreated when the backend starts;
- changes remain visible while the backend is running;
- changes survive frontend page refreshes;
- all runtime changes disappear after backend shutdown or restart.

Avatar files are stored separately in the local file system.